

.NET MİMARİ REHBERİ

CQRS, Onion Architecture ve Code First

Tasarım desenleri ve mimari yaklaşımlar
— sıfırdan, örneklerle

Her terim bilinmediği varsayılarak açıklanmıştır.

Gerçek hayat benzetmeleri ve C# kod örnekleriyle.

Ağustos 2026

İçindekiler

Bölüm 0 — Bu Rehber Nasıl Okunur	4
Ortak metaforumuz: Bir lokanta.....	4
Neden bu kadar uğraşıyoruz?	5
Bölüm 1 — Temel Kavramlar	6
1.1 CRUD	6
1.2 Ön bilgi: Assembly, CLR ve ORM.....	6
1.3 POCO (Plain Old CLR Object)	7
1.4 DTO (Data Transfer Object).....	7
1.5 Mapping (Eşleme)	9
1.6 Validator (Doğrulayıcı)	9
1.7 Use Case (Kullanım Senaryosu)	11
1.8 Web API	12
Bölüm 2 — Veritabanı Tarafı: Code First.....	14
2.1 Üç yaklaşım.....	14
2.2 DbContext	14
2.3 Fluent API ve Data Annotation	16
2.4 Composite Key (Bileşik Anahtar)	19
2.5 Migration (Göç)	21
2.6 SaveChanges'i özelleştirmek.....	22
2.7 CI/CD	23
Bölüm 3 — Katmanlar ve Aralarındaki İletişim.....	26
3.1 Onion Architecture	26
3.2 Dependency Inversion Principle	29
3.3 Dependency Injection (Bağımlılık Enjeksiyonu).....	32
3.4 WebAPI'nin referans problemi ve çözümü	33
Bölüm 4 — CQRS ve Handler Dünyası	37
4.1 CQS'ten CQRS'e	37
4.2 MediatR.....	38
4.3 Handler'ın anatomisi	41
4.4 Query handler ve projection	44
4.5 record, immutability ve pipeline	46
4.6 Pipeline Behavior — detaylı	49
4.7 Notification (Domain Event) — bire çok	52

Bölüm 5 — Bir İsteğin Uçtan Uca Yolculuğu	54
Adım adım	54
Bölüm 6 — Design Pattern Sözlüğü	57
6.1 Repository Pattern	57
6.2 Unit of Work	58
6.3 Specification Pattern	58
6.4 Result Pattern	58
6.5 Diğer desenler	59
Bölüm 7 — Mimari Yaklaşımlar	63
7.1 N-Tier / Layered	63
7.2 Hexagonal (Ports & Adapters)	63
7.3 Clean Architecture	63
7.4 Vertical Slice Architecture	63
7.5 Modular Monolith	64
7.6 Microservices	64
Bölüm 8 — Proje İskeleti	65
Referans yönleri	66
Mimari testler	66
Bölüm 9 — Ne Zaman Ne Kullanmalı?	67
Karar tablosu	67
Sağlıklı ilerleme sırası	67
Ek A — Terimler Sözlüğü	68
Ek B — Sık Yapılan Hatalar	72

Bölüm 0 — Bu Rehber Nasıl Okunur

Bu rehber, .NET dünyasında sıkça duyduğun ama çoğu kaynağın "zaten biliyorsun" varsayarak geçtiği kavramları sıfırdan anlatır. Hiçbir terimi bildiğin varsayılmaz; bir terim geçtiğinde önce o terim açıklanır, sonra konuya devam edilir.

Üç şey birlikte gider:

1. **Gerçek hayat benzetmesi** — kavramın neye benzediği
2. **Problem** — bu kavram olmasaydı başımıza ne gelirdi
3. **Kod** — .NET'te pratikte nasıl görüldüğü

Ortak metaforumuz: Bir lokanta

Baştan sona aynı benzetmeyi kullanacağız. Yazılım katmanları, bir lokantanın bölümleri gibi çalışır: herkesin tek bir işi vardır ve kimse başkasının işine karışmaz.

Lokantada	Yazılımda	Görevi
Müşteri	Kullanıcı, tarayıcı, mobil uygulama	İstekte bulunur
Menü	API sözleşmesi	Nelerin istenebileceğini söyler
Garson	Controller (Web API)	İsteği alır, mutfağa iletir, tabağı getirir
Sipariş fişi	Command / Query	Ne istendiğini yazan kâğıt
Kasadaki fiş kontrolcüsü	Validator	Eksik doldurulmuş fişi geri çevirir
Mutfak koordinatörü	MediatR	Fişi okur, doğru aşçıya verir
Aşçı	Handler	O siparişi baştan sona hazırlar
Mutfak kuralları, tarif kitabı	Domain	"Tavuk 74 derecenin altında servis edilmez"
Depo görevlisi	Repository	Malzemeyi depodan getirir, depoya koyar
Depo	Veritabanı	Malzemenin saklandığı yer
Akşam sayım defteri	DbContext / SaveChanges	Gün içindeki değişiklikleri tek seferde işler
Tabak	DTO	Müşteriye gösterilecek, sunuma hazır hâl

Bu tabloya rehber boyunca defalarca döneceğiz. Şimdilik sadece şu fikri aklında tut: **lokantada müşteri mutfağa girmez, aşçı da salona çıkıp sipariş almaz**. Katmanlı mimarinin bütün amacı budur.

Neden bu kadar uğraşıyoruz?

Dürüst cevap: **yazılım projelerinin çoğu zamanla ölür**. Ölüm sebebi genelde şudur — bir şeyi değiştirmek istersin, ama o şey başka on yere dokunmuştur. Veritabanını değiştirmek istersin, iş kuralların çöker. E-posta sağlayıcısını değiştirmek istersin, sipariş kodunu açman gerekir. Bir hata bulursun, düzeltirsin, üç yeni hata çıkar.

Bu rehberdeki her kavram, tek bir soruya verilmiş cevaptır:

Bir şeyi değiştirdiğimde başka neyi değiştirmek zorunda kalacağım?

İyi mimari bu sayıyı küçültür. Hepsi bu. Kalan her şey detaydır.

Bölüm 1 — Temel Kavramlar

1.1 CRUD

Açılımı **Create, Read, Update, Delete**. Türkçesi: Oluştur, Oku, Güncelle, Sil.

Bu dört fiil, veriyle yapabileceğin neredeyse her şeyin özetidir. Telefon rehberini düşün: yeni kişi eklersin (Create), kişiyi ararsın (Read), numarasını değiştirirsin (Update), kişiyi silersin (Delete). Beşinci bir şey yoktur.

Terim, mimari kararlarında bir sınıflandırma aracı olarak kullanılır:

"**Bu bir CRUD uygulaması**" demek → içinde ciddi iş kuralı yok, veri sadece tabloya yazılıp okunuyor. Bir blog admin paneli, bir kategori yönetim ekranı, bir ayarlar sayfası genelde budur.

"**Bu CRUD değil**" demek → verinin değişmesi bir sürü kurala tabi. Banka havalesi CRUD değildir: bakiye yetiyor mu, günlük limit aşıldı mı, alıcı hesap kapalı mı, aynı işlem daha önce yapılmış mı?

Neden önemli? Mimari kararını buna göre verirsin. Saf CRUD ekranları için Onion + CQRS + Repository + Unit of Work kurmak, çivi çakmak için vinç kiralamaktır. Aynı projede bazı modüller CRUD, bazıları karmaşık olabilir — ve bunlara farklı davranmak tamamen meşrudur.

1.2 Ön bilgi: Assembly, CLR ve ORM

Aşağıdaki üç terim rehber boyunca sürekli geçecek, o yüzden burada halledelim.

CLR (Common Language Runtime)

C# kodun doğrudan işlemcide çalışmaz. Önce **IL** (Intermediate Language) denen ara bir dile derlenir, sonra CLR adı verilen motor bunu çalışma anında makine koduna çevirir ve yürütür. Java dünyasındaki JVM'in birebir karşılığıdır.

CLR'nin sana sağladıkları: bellek yönetimi (çöp toplayıcı / garbage collector), tip güvenliği, istisna yönetimi ve **reflection** (birazdan geliyor).

Assembly

Derlenmiş bir .NET projesinin çıktısı. Pratikte bir **.dll** veya **.exe** dosyasıdır.

Bu rehberde "katman" dediğimiz şeylerin her biri (**Domain**, **Application**, **Infrastructure**, **WebAPI**) ayrı bir projedir ve derlendiğinde ayrı bir assembly üretir. Katmanlar arası kuralların derleyici tarafından zorlanabilmesinin sebebi budur: **Domain.dll** içinde **Infrastructure.dll**'e referans yoksa, oradaki bir sınıfı kullanman **fiziksel olarak imkânsızdır**. Kural sadece bir tavsiye değil, derleme hatasıdır.

ORM (Object-Relational Mapper)

Ortada bir uyumsuzluk vardır: veritabanı satır ve kolonlardan oluşur, C# ise nesnelerden. Buna literatürde **impedance mismatch** (empedans uyumsuzluğu) denir.

ORM bu iki dünya arasında tercümanlık yapar. Sen `product.Name = "Klavye"` yazarsın, ORM `UPDATE Products SET Name='Klavye' WHERE Id=@id` SQL'ini üretir. .NET'te en yaygın ORM **Entity Framework Core**'dur (kısaca EF Core).

1.3 POCO (Plain Old CLR Object)

Kelime kelime: *Plain Old* = "sade, eski usul", *CLR* = .NET motoru, *Object* = nesne.

Tanım: hiçbir kütüphaneye bulaşmamış, sade bir C# sınıfı.

Neden ayrı bir isim verilmiş? Çünkü eskiden ORM'ler ve framework'ler senden sınıflarını kendi taban sınıflarından türetmeni isterdi:

```
// POCO DEĞİL - kütüphaneye zincirlenmiş
public class Product : EntityObject, ISerializable, ITrackable
{
    // Bu sınıfı başka bir projede kullanamazsın,
    // çünkü EntityObject'i ve tüm bağımlılıklarını da taşıman gerekir.
    // Birim testinde bu sınıfı yaratmak için framework'ü ayağa kaldırman gerekir.
}
```

```
// POCO - tertemiz
public class Product
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Gerçek hayat karşılığı. Tarif kitabındaki "menemen tarifi" sadece bir tariftir. İçinde "Arçelik marka ocakta pişirilir" yazmaz. Ocağın markası değiştiğinde tarif hâlâ geçerlidir. POCO da böyledir: veritabanı SQL Server'dan PostgreSQL'e geçtiğinde `Product` sınıfına dokunmazsın.

Dikkat. POCO olmak "hiç property'si olmayan" demek değildir. `Guid`, `string`, `DateTime` gibi `System` altındaki temel tipleri kullanmak POCO'luğu bozmaz — onlar dilin kendi parçasıdır. Bozan şey, dışarıdan gelen bir NuGet paketine bağımlı olmaktır.

1.4 DTO (Data Transfer Object)

Türkçesi: Veri Taşıma Nesnesi. Sadece veri taşımak için var olan, içinde hiçbir davranış bulunmayan sınıf.

İki ayrı problemi çözer.

Problem 1: Güvenlik

```
// Domain'deki gerçek nesne - İÇERİDE kalmalı
public class User
{
    public Guid Id { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; } // Bu dışarı çıkarsa felaket
    public decimal Salary { get; set; } // Bu da
    public bool IsAdmin { get; set; }
    public List<Order> Orders { get; set; }
}
```

Bu nesneyi doğrudan API'den döndürseydin, kullanıcının şifre hash'i ve maaşı JSON'a gidecekti. Üstelik **Orders** listesi yüzünden yüzlerce siparişi de beraberinde çekmeye çalışacaktı.

```
// DTO - DIŞARI verilecek versiyon, sadece gerekli alanlar
public sealed record UserDto(Guid Id, string Email, string FullName);
```

Problem 2: Kırılganlık



Domain nesneni doğrudan döndürüyorsan, içeride yaptığın en küçük değişiklik dışarıyı kırar. **Name** property'sini **FullName** yapmaya karar verdin — mobil uygulama patladı, çünkü JSON alan adı değişti.

DTO araya girdiğinde bu bağ kopar. İçeride istediğini değiştirir, mapping katmanında düzeltir, dışarıya aynı şekli sunmaya devam edersin.

Gerçek hayat karşılığı. Lokantada müşteriye tencerenin kendisini götürmezsin. Tabağa, sunum için gereken kadarını koyup götürürsün. Tencere = domain entity, tabak = DTO. Müşterinin tencerede kaç porsiyon kaldığını görmesine gerek yoktur — hatta görmemesi gerekir.



DTO çeşitleri

Tip	Yön	Örnek
Request DTO / Command	Dışarıdan içeri	CreateProductCommand
Response DTO	İçeriden dışarı	ProductDetailDto
List item DTO	İçeriden dışarı, liste için	ProductListItemDto (daha az alan)
View Model	Belirli bir ekran için şekillendirilmiş	ProductEditPageViewModel

Liste ve detay için ayrı DTO yazmak boşuna görünebilir ama değildir: liste ekranında 4 kolon gösteriyorsan, veritabanından 25 kolon çekmenin hiçbir anlamı yoktur.

1.5 Mapping (Eşleme) ✂

Product nesnesindeki verileri ProductDto nesnesine kopyalama işidir.

```
// Elle mapping - en hızlısı, en açık olanı, hata yaparsan derleyici söyler
var dto = new ProductDto(
    Id: product.Id,
    Name: product.Name,
    Price: product.Price.Amount,
    CategoryName: product.Category.Name
);
```

30 sınıfın ve her birinin 12 property'si varsa bu iş yorucu olur. Kütüphaneler devreye girer:

```
// Mapper - konfigürasyonsuz, isim eşleşmesine göre
var dto = product.Adapt<ProductDto>();

// AutoMapper - önce profil tanımlarsın
public class ProductProfile : Profile
{
    public ProductProfile()
    {
        CreateMap<Product, ProductDto>()
            .ForMember(d => d.CategoryName, o => o.MapFrom(s => s.Category.Name));
    }
}
```

Dikkat — otomatik mapping'in bedeli. Elle yazdığın mapping'de bir alanı unutursan derleyici hata verir. Otomatik mapping'de unutilan alan sessizce `null` kalır ve hatayı üretimde, müşteri şikâyet ettiğinde öğrenirsin. Ayrıca profil dosyaları zamanla kimsenin anlamadığı bir konfigürasyon yumağına dönüşme eğilimindedir.

Tavsiye: Küçük ve orta projede elle yaz. Tekrar gerçekten canını yakmaya başladığında kütüphane getir. Getirdiğinde de mapping'lerin doğruluğunu test ile doğrula (AutoMapper'ın `AssertConfigurationIsValid()` metodu tam bunun içindir).

1.6 Validator (Doğrulayıcı)

Gelen verinin şeklen doğru olup olmadığını kontrol eden sınıf. .NET'te en yaygın kütüphane FluentValidation'dır.

```

public sealed class CreateProductCommandValidator : AbstractValidator<CreateProductCommand>
{
    public CreateProductCommandValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("Ürün adı zorunludur.")
            .MaxLength(200).WithMessage("Ürün adı 200 karakteri geçemez.");

        RuleFor(x => x.Price)
            .GreaterThan(0).WithMessage("Fiyat sıfırdan büyük olmalıdır.");

        RuleFor(x => x.Currency)
            .NotEmpty()
            .Length(3).WithMessage("Para birimi 3 harfli olmalıdır (TRY, USD, EUR).");

        RuleFor(x => x.Stock)
            .GreaterThanOrEqualTo(0).WithMessage("Stok negatif olamaz.");
    }
}

```

Ara bilgi: Lambda ifadesi

`RuleFor(x => x.Name)` içindeki `x => x.Name` bir **lambda ifadesidir**. "Bana bir `x` ver, ben sana onun `Name`'ini vereyim" demektir. İsimsiz, tek satırlık bir fonksiyondur.

Property adını string olarak (`RuleFor("Name")`) yazmak yerine böyle yazmanın avantajı: property'nin adını değiştirdiğinde derleyici seni uyarır. String yazsaydın, kod sessizce çalışmaya devam eder ama kural hiç uygulanmazdı.

Kritik ayırım: Nerede ne kontrol edilir?

Bu, yeni başlayanların en sık karıştırdığı konudur. Üç ayrı yer vardır ve her birinin görevi farklıdır:

Soru	Nerede kontrol edilir	Neden orada?
Fiyat negatif mi?	Validator	Şekil kontrolü, hiçbir bilgi gerektirmez
E-posta formatı doğru mu?	Validator	Aynı sebep
Zorunlu alan boş mu?	Validator	Aynı sebep
Bu kategori sistemde var mı?	Handler	Veritabanına bakmak gerekir
Bu kullanıcının yetkisi var mı?	Handler veya yetki behavior'ı	Bağlam bilgisi gerekir
Stok bu siparişe yetiyor mu?	Entity'nin içi	Nesnenin kendi durumuna bağlı iş kuralı
Sipariş zaten iptal edilmiş mi?	Entity'nin içi	Aynı sebep

Gerçek hayat karşılığı. Validator, kasadaki fiş kontrolçüsüdür. "Bu fişte masa numarası yazmamış, adet hanesi boş kalmış" der ve fişi mutfağa **hiç göndermez**. Ama "bu yemeğin malzemesi bitti mi" sorusuna cevap veremez — deponun ne durumda olduğunu bilmez. O mutfağın işidir.

1.7 Use Case (Kullanım Senaryosu)



Sistemin dışarıya sunduğu **tek bir anlamlı iş**. Kullanıcının yapmak istediği şey.



Use case olanlar:

- Ürün oluştur
- Siparişi iptal et
- Şifremi unuttum e-postası gönder
- Aylık satış raporunu indir



Use case olmayanlar:

- Veritabanına bağlan (*teknik detay, kullanıcının amacı değil*)
- **Products** tablosuna INSERT at (*bir adım, hedef değil*)
- Cache'i temizle (*bir yan işlem*)

Basit test. Bir işi teknik kelime kullanmadan patronuna anlatabiliyorsan, o bir use case'tir. "Müşteri siparişini iptal edebilsin" → use case. "EF Core'un change tracker'ını temizle" → use case değil.

Screaming Architecture

Robert C. Martin'in ortaya attığı bir fikir: **mimari, uygulamanın ne yaptığını bağırmalıdır**, hangi framework'le yazıldığını değil.

Bir kütüphane binasına girdiğinde kitaplar, raflar, okuma masaları görürsün ve "burası bir kütüphane" dersin. Binanın hangi tuğlayla örüldüğünü ilk bakışta anlamazsın ve anlamana gerek yoktur. Klasör yapın da böyle olmalıdır:

Teknoloji bağırıyor:

Controllers/
Services/
Repositories/
Models/
Helpers/

İş bağırıyor:

Features/
Products/
CreateProduct/
DecreaseStock/
GetProductList/
Orders/
PlaceOrder/
CancelOrder/
Payments/
RefundPayment/

Soldaki yapıda "sipariş iptali nerede yazıyor?" sorusunun cevabı yoktur — dört ayrı klasöre bakman gerekir. Sağdaki yapıda `Orders/CancelOrder/` klasörünü açarsın, her şey oradadır.

1.8 Web API

API nedir?

Application Programming Interface — "programlar arası arayüz". Bir programın dışarıya açtığı kapı. Sen o kapıdan istek atarsın, program sana cevap verir. İçinin nasıl çalıştığını bilmen gerekmez.

Gerçek hayat karşılığı. Restoranın menüsü bir API'dir. Menüde "42 numara: Karnıyarık" yazar. Sen "42" dersin, karnıyarık gelir. Mutfakta patlıcanın nasıl kızartıldığını, hangi markanın yağının kullanıldığını bilmen gerekmez. Menü = sözleşme (contract), mutfak = implementasyon (uygulama).

Menünün en güzel tarafı: mutfak tamamen yenilense, aşçı değişse, ocaklar değişse bile menü aynı kaldığı sürece senin için hiçbir şey değişmez.

Web API

Bu kapının HTTP protokolü üzerinden, ağ üzerinden açılmış hâli. Pratikte şöyle görünür:

```
[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase
{
    [HttpGet("{id}")] // GET /api/products/5
    public async Task<IActionResult> GetById(Guid id) { }

    [HttpPost] // POST /api/products
    public async Task<IActionResult> Create(CreateProductCommand cmd) { }

    [HttpPut("{id}")] // PUT /api/products/5
    public async Task<IActionResult> Update(Guid id, UpdateProductCommand cmd) { }

    [HttpDelete("{id}")] // DELETE /api/products/5
    public async Task<IActionResult> Delete(Guid id) { }
}
```

Ara bilgi: Attribute nedir?

Köşeli parantez içindeki `[HttpGet]`, `[Route]`, `[ApiController]` gibi ifadeler **attribute** (öznitelik) denir. Sınıflara, metotlara veya property'lere yapıştırılan etiketlerdir. Kendi başlarına hiçbir şey yapmazlar — bir başkasının onları okuyup davranış değiştirmesi için oradadırlar.

`[HttpGet]` etiketi ASP.NET Core'a "bu metoda GET isteğiyle ulaşılır" der. ASP.NET Core başlangıçta tüm controller'ları tarar, bu etiketleri okur ve bir yönlendirme tablosu oluşturur.



HTTP fiilleri ve CRUD karşılıkları

HTTP fiili	CRUD	Anlamı	Güvenli mi?
GET	Read	Getir, hiçbir şeyi değiştirme	Evet
POST	Create	Yeni kayıt oluştur	Hayır
PUT	Update	Kaydın tamamını değiştir	Hayır
PATCH	Update	Kaydın bir kısmını değiştir	Hayır
DELETE	Delete	Sil	Hayır

"Güvenli" (safe) burada teknik bir terimdir: isteği kaç kez tekrarlırsan tekrarla sistemin durumu değişmiyorsa güvenlidir. Bu yüzden tarayıcılar GET isteklerini ön belleğe alabilir, POST isteklerini alamaz.

JSON, serialization, deserialization

Veri ağ üzerinden giderken **JSON** (JavaScript Object Notation) formatına çevrilir. Hem insanın hem makinenin okuyabildiği bir metin formatıdır:

```
{
  "id": "a3f2c891-4d5e-4f6a-8b7c-9d0e1f2a3b4c",
  "name": "Mekanik Klavye",
  "price": 2450.00,
  "categoryName": "Bilgisayar Aksesuarları"
}
```

- C# nesnesini JSON metnine çevirmeye **serialization** (serileştirme) denir.
- JSON metnini tekrar C# nesnesine çevirmeye **deserialization** (ters serileştirme) denir.

ASP.NET Core bunu senin için otomatik yapar. Controller metodun `CreateProductCommand` tipinde bir parametre bekliyorsa, gelen JSON gövdesi otomatik olarak o nesneye dönüştürülür. Bu işleme **model binding** denir.

Bölüm 2 — Veritabanı Tarafı: Code First

2.1 Üç yaklaşım

Veritabanı ile kodun arasındaki ilişkiyi kurmanın üç yolu vardır. Aradaki fark tek bir soruda düğümlenir: **doğrunun kaynağı hangisi?**

Yaklaşım	Kaynak doğruluk	Nasıl çalışır	Kimler kullanır
Code First	C# sınıfları	Sınıfları yazarsın, tablolar üretilir	Yeni projeler, DDD, modern .NET
Database First	Mevcut veritabanı	Veritabanından sınıflar üretilir	Legacy sistemler, DBA'nın DB'yi yönettiği kurumlar
Model First	Görsel EDMX tasarımcı	Diagram çizersin, ikisi de üretilir	Neredeyse ölü; EF Core desteklemiyor

Code First'te önce POCO sınıflarını yazarsın, EF Core senin için tabloları üretir. Bu rehberde Code First'e odaklanıyoruz.

Code First neden tercih ediliyor?

1. **Sürüm kontrolü.** Şema değişiklikleri migration dosyaları olarak Git'e girer. "Bu kolon ne zaman, kim tarafından, neden eklendi?" sorusunun cevabı commit geçmişindedir.
2. **İş kuralları önce gelir.** Önce "sipariş nedir, ne yapılabilir" sorusunu cevaplarsın; tablo yapısı bunun sonucu olur. Tersi durumda tablo yapısı iş mantığına şekil verir ve bu genelde kötü bir modelle sonuçlanır.
3. **Takım hızı.** Herkes kendi makinesinde `dotnet ef database update` diyerek güncel şemaya ulaşır. Kimsenin kimseye SQL script'i göndermesi gerekmez.

2.2 DbContext

DbContext, EF Core'un ana sınıfıdır. Üç ayrı işi vardır.

```

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    // İş 1: Hangi tablolar var?
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Order> Orders => Set<Order>();

    // İş 2: Tablolar nasıl kurulacak?
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
        base.OnModelCreating(modelBuilder);
    }
}

```

İş 3: Change Tracker — en sihirli kısım

```

// 1. Ürünü veritabanından çektik
var product = await _context.Products.FirstAsync(p => p.Id == id);

// 2. Sadece C# nesnesinin bir property'sini değiştirdik
product.Name = "Yeni İsim";

// 3. Hiçbir UPDATE cümlesi yazmadık, ama...
await _context.SaveChangesAsync();

// EF Core arka planda şunu çalıştırdı:
// UPDATE Products SET Name = 'Yeni İsim' WHERE Id = @id

```

Nasıl bildi? Çünkü `DbContext` nesneyi veritabanından çektiğinde onun bir **anlık görüntüsünü** (snapshot) hafızada saklar. `SaveChangesAsync()` çağrıldığında elindeki nesneyi bu görüntüyle karşılaştırır, farkları bulur ve sadece değişen kolonlar için SQL üretir.

Gerçek hayat karşılığı. Bir depo görevlisi düşün. Sabah deponun sayımını yapıp defterine yazıyor. Gün boyu aşçılar malzeme alıp koyuyor, kimse ona haber vermiyor. Akşam görevli defteriyle depoyu karşılaştırıyor: "3 kilo un eksilmiş, 2 kasa domates eklenmiş" diye tek seferde ana kayda işliyor.

`SaveChangesAsync()` = akşam sayımı. Gün boyu yapılan onlarca değişiklik tek bir işlemde, tek bir transaction içinde veritabanına yansır.

Takibin bedeli ve AsNoTracking

Bu takip bedava değildir. Her nesne için hafızada bir kopya tutulur, `SaveChanges` sırasında her biri karşılaştırılır. 5000 satır çekip listeleyeceksen bu tamamen israftır.

```
// Sadece listeleyeceğim, değiştirmeyeceğim → takibi kapat
var products = await _context.Products
    .AsNoTracking()
    .ToListAsync();
```

`AsNoTracking()` kullanmak, okuma ağırlıklı sorgularda gözle görülür bir hız farkı yaratır. **Kural basit:** veriyi değiştirmeyeceksen takip etme.

Önemli fark ediş. `DbContext` iki tasarım desenini zaten içinde barındırır:

- Her `DbSet<T>` bir **Repository**'dir — bellek içi bir koleksiyon gibi davranır, `Add`, `Remove`, LINQ sorguları alır.
- `SaveChangesAsync()` bir **Unit of Work**'tür — birden çok değişikliği tek atomik işlemde kaydeder.

Bu, ilerideki "Repository Pattern gerçekten gerekli mi?" tartışmasının kökenidir. Zaten var olan bir şeyin üstüne bir katman daha koyuyorsan, bunun ne kazandırdığını net cevaplayabilmen gerekir.

DbContext'in yaşam süresi

`DbContext` **kısa ömürlü** olmalıdır. Bir HTTP isteği boyunca yaşar, istek bitince yok edilir. Bu yüzden DI'ye `Scoped` olarak kaydedilir:

```
services.AddDbContext<AppDbContext>(opt =>
    opt.UseSqlServer(configuration.GetConnectionString("Default")));
// AddDbContext varsayılan olarak Scoped kaydeder
```

Uzun ömürlü bir `DbContext` iki sorun yaratır: change tracker sonsuza kadar şişer (bellek sızıntısı), ve iki farklı işlem aynı nesneler üzerinde çalışmaya başlar (veri tutarsızlığı).

2.3 Fluent API ve Data Annotation

EF Core'a "bu tablo nasıl olsun" demenin iki yolu vardır.

Yol 1: Data Annotation

Kuralları doğrudan sınıfın üstüne etiket olarak yapıştırırsın.


```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("Products")]
public class Product
{
    [Key]
    public Guid Id { get; set; }

    [Required]
    [MaxLength(200)]
    public string Name { get; set; }

    [Column(TypeName = "decimal(18,2)")]
    public decimal Price { get; set; }
}

```

Kolay ve okunaklıdır. İki sorunu vardır:

1. **Sınıfı kirletir.** `Product` artık POCO değildir; `System.ComponentModel.DataAnnotations` kütüphanesini tanımak zorundadır. Domain katmanının "hiçbir şey bilmemeli" kuralını çiğnedin.
2. **Yetersiz kalır.** Composite key, ilişki silme davranışı, global sorgu filtresi, value object gömme, index tanımlama — bunların hiçbiri attribute ile yapılamaz.

Yol 2: Fluent API

Fluent (akıcı) kelimesi, metotların birbirine zincirlenerek cümle gibi okunmasından gelir:

```

builder.Property(p => p.Name).IsRequired().HasMaxLength(200);
//      "property Name, zorunlu, maksimum 200 karakter"

```

Buna **method chaining** (metot zincirleme) denir. Her metot işini yapıp `this`'i (kendisini) döndürdüğü için nokta koyup devam edebilirsin.

Kuralları sınıfın içine değil, ayrı bir konfigürasyon dosyasına yazarsın:

```
// Infrastructure/Persistence/Configurations/ProductConfiguration.cs
public class ProductConfiguration : IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> builder)
    {
        builder.ToTable("Products");
        builder.HasKey(p => p.Id);

        builder.Property(p => p.Name)
            .IsRequired()
            .HasMaxLength(200);

        builder.Property(p => p.Price)
            .HasPrecision(18, 2);

        // İlişki tanımı:
        builder.HasOne(p => p.Category)           // Ürünün BİR kategorisi var
            .WithMany(c => c.Products)           // Kategorinin ÇOK ürünü var
            .HasForeignKey(p => p.CategoryId)
            .OnDelete(DeleteBehavior.Restrict); // Kategori silinirse ürünleri SİLME, hata
ver

        builder.HasIndex(p => p.Name);           // Arama sorguları hızlansın

        // Global filtre: silinmiş kayıtlar hiçbir sorguda görünmesin
        builder.HasQueryFilter(p => !p.IsDeleted);
    }
}
```

Bu dosyalar **Infrastructure** katmanında durur. **Product** sınıfı **Domain**'de tertemiz bir POCO olarak kalır. **DbContext** hepsini tek satırla toplar:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Bu assembly'deki tüm IEntityTypeConfiguration<> sınıflarını bul ve uygula
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}
```

Gerçek hayat karşılığı. Data Annotation, yemeğin kabına "tuzsuz pişecek" yazan bir etiket yapıştırmaktır — etiket yemekle birlikte gezer, müşterinin önüne kadar gider.

Fluent API ise bu notu mutfağın duvarındaki tarif panosuna yazmaktır. Yemek yine yemektir, kurallar mutfakta kalır. Yemek başka bir mutfağa taşındığında etiketi kazımak zorunda değilsin.

Onları ne zaman ayırt edeceksin?

Özellik	Data Annotation	Fluent API
Zorunlu alan, uzunluk	✓	✓
Basit primary key	✓	✓
Composite key	✗	✓
İlişki silme davranışı	✗	✓
Global query filter	✗	✓
Value object gömme (owned type)	✗	✓
Index tanımlama	Kısıtlı	✓
Domain'i temiz bırakır	✗	✓

İkisini aynı anda kullanabilirsin; çakışarlarsa **Fluent API kazanır**.

2.4 Composite Key (Bileşik Anahtar)

Önce: Primary key nedir?

Bir tablodaki her satırı benzersiz şekilde tanımlayan kolon. Kimlik numarası gibi — iki satırın aynı primary key'i olamaz.

Composite key

Bu görevi **birden fazla kolonun birlikte** üstlenmesidir. Tek başına hiçbirisi benzersiz değildir, ama bir aradayken benzersizdir.

Gerçek hayat karşılığı — sinema salonu.

- Sıra numarası tek başına benzersiz değil: 5. sırada 20 koltuk var.
- Koltuk numarası tek başına benzersiz değil: her sırada bir 3 numaralı koltuk var.
- Ama **(Sıra 5, Koltuk 3)** ikilisi salondaki tek bir koltuğu işaret eder.

İşte bu ikili, composite key'dir.

Nerede karşına çıkar? Çoktan çoğa ilişki

Bir öğrencinin çok dersi, bir dersin çok öğrencisi vardır. Bunu iki tabloyla ifade edemezsin; araya bir **bağlantı tablosu** (junction / join table) koyarsın:

```
public class StudentCourse
{
    public Guid StudentId { get; set; }
    public Student Student { get; set; }

    public Guid CourseId { get; set; }
    public Course Course { get; set; }

    // İlişkiye ait ek bilgiler
    public DateTime EnrolledAt { get; set; }
    public int? Grade { get; set; }
}
```

```
public class StudentCourseConfiguration : IEntityTypeConfiguration<StudentCourse>
{
    public void Configure(EntityTypeBuilder<StudentCourse> builder)
    {
        // İşte composite key: iki kolon BİRLİKTE anahtar
        builder.HasKey(sc => new { sc.StudentId, sc.CourseId });

        builder.HasOne(sc => sc.Student)
            .WithMany(s => s.StudentCourses)
            .HasForeignKey(sc => sc.StudentId);

        builder.HasOne(sc => sc.Course)
            .WithMany(c => c.StudentCourses)
            .HasForeignKey(sc => sc.CourseId);
    }
}
```

Ara bilgi: Anonim tip

`new { sc.StudentId, sc.CourseId }` ifadesindeki `new { ... }` bir **anonim tiptir** — adı olmayan, o an oluşturulan bir nesne. Sadece birkaç değeri bir arada taşımak için kullanılır.

EF Core buradan "bu iki property birlikte anahtar oluşturuyor" anlamını çıkarır.

Bedava gelen bir iş kuralı

Composite key'in güzel bir yan etkisi vardır: **aynı öğrenci aynı derse iki kez kaydolamaz**. Çünkü veritabanı aynı anahtarları iki kez kabul etmez. İş kuralını en alt seviyede, veritabanı tarafından garanti altına almış olursun. Uygulamanda bir hata olsa bile veritabanı bu duruma izin vermez.

Composite key, `[Key]` attribute'u ile **yapılamaz**. Fluent API'nin neden gerekli olduğunun en net örneğidir.

2.5 Migration (Göç)



Migration, veritabanı şemasındaki bir değişikliği tarif eden, sürüm kontrolüne giren C# dosyasıdır.

Temel komutlar

```
# Yeni migration oluştur
dotnet ef migrations add InitialCreate -p Infrastructure -s WebAPI

# Veritabanına uygula
dotnet ef database update -p Infrastructure -s WebAPI

# Henüz uygulanmamış son migration'ı geri al
dotnet ef migrations remove -p Infrastructure -s WebAPI

# Belirli bir migration'a geri dön
dotnet ef database update PreviousMigrationName -p Infrastructure -s WebAPI

# Üretim için idempotent SQL script üret
dotnet ef migrations script --idempotent --output migrate.sql -p Infrastructure -s WebAPI
```

Parametreler:

- **-p (project)** → migration dosyalarının yazılacağı proje. **DbContext** neredeyse orası.
- **-s (startup project)** → uygulamanın giriş noktası. Connection string'in okunacağı yer (**appsettings.json** orada).

Üretilen dosya nasıl görünür?

```
public partial class AddProductStock : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<int>(
            name: "Stock",
            table: "Products",
            nullable: false,
            defaultValue: 0);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(name: "Stock", table: "Products");
    }
}
```

Up ileri gider, **Down** geri alır. EF Core veritabanında **EFMigrationsHistory** adlı bir tablo tutar ve hangi migration'ların uygulandığını orada saklar.

Üretimde Database.Migrate() çağırma.

Birçok örnek kodda Program.cs içinde şunu görürsün:

```
app.Services.GetRequiredService<AppDbContext>().Database.Migrate();
```

Geliştirme ortamında pratiktir, ama üretimde tehlikelidir. Uygulaman yük dengeleme için 3 kopya hâlinde çalışıyorsa, üçü aynı anda ayağa kalkar ve üçü birden şemayı değiştirmeye çalışır. Sonuç: kilitlenme veya yarım kalmış migration.

Doğrusu: migration'ı CI/CD hattında, dağıtımdan önce, tek seferde çalıştırmak.

2.6 SaveChanges'i özelleştirmek

Code First'ün en güçlü taraflarından biri, **SaveChangesAsync**'i ezerek (override ederek) tüm kayıt işlemlerine merkezî davranış eklemektir.

```
public override async Task<int> SaveChangesAsync(CancellationToken ct = default)
{
    foreach (var entry in ChangeTracker.Entries<BaseEntity>())
    {
        switch (entry.State)
        {
            case EntityState.Added:
                entry.Entity.CreatedAt = _dateTime.UtcNow;
                entry.Entity.CreatedBy = _currentUser.UserId;
                break;

            case EntityState.Modified:
                entry.Entity.UpdatedAt = _dateTime.UtcNow;
                entry.Entity.UpdatedBy = _currentUser.UserId;
                break;

            case EntityState.Deleted:
                // Soft delete: gerçekten silme, sadece işaretle
                entry.State = EntityState.Modified;
                entry.Entity.IsDeleted = true;
                entry.Entity.DeletedAt = _dateTime.UtcNow;
                break;
        }
    }

    var result = await base.SaveChangesAsync(ct);

    // Domain event'leri veritabanı işlemi BAŞARILI olduktan SONRA yayınla
    await _domainEventDispatcher.DispatchAsync(ChangeTracker, ct);

    return result;
}
```

Bu 25 satır sayesinde:

- Hiçbir handler'da `CreatedAt = DateTime.Now` yazmana gerek kalmaz
- Kimse yanlışlıkla `CreatedBy` doldurmayı unutamaz
- Silme işlemi otomatik olarak soft delete'e döner (`HasQueryFilter` ile birleşince silinen kayıtlar hiçbir sorguda görünmez)

`DateTime.Now` yerine `DateTime` servisi. Yukarıdaki kodda saati doğrudan `DateTime.UtcNow` ile almak yerine enjekte edilmiş bir servisten alıyoruz. Sebep: test edilebilirlik. Testte "şu an 2026-01-15 saat 10:00" diyebilmek için saatin de bir bağımlılık olması gerekir. Bu, bir sonraki bölümdeki DIP prensibinin küçük ama çok pratik bir uygulamasıdır.

2.7 CI/CD

- **CI** = Continuous Integration (Sürekli Entegrasyon)
- **CD** = Continuous Delivery / Deployment (Sürekli Teslimat / Dağıtım)

Çözdüğü problem

Eski dünyada şöyleydi: 5 geliştirici 3 ay boyunca ayrı ayrı çalışır, sonunda kodları birleştirmeye çalışırlar ve ortalık savaş alanına döner. Buna **merge hell** (birleştirme cehennemi) denirdi. Herkesin kodu ayrı ayrı çalışıyordu ama bir araya gelince hiçbiri çalışmıyordu.

CI'nin çözümü

Herkes kodunu günde birkaç kez ana dala gönderir ve **her gönderimde otomatik olarak** derleme ve testler çalışır:

```
# .github/workflows/ci.yml - GitHub Actions örneği
name: CI

on:
  push:
    branches: [ main, develop ]
  pull_request:

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # Kodu indir

      - uses: actions/setup-dotnet@v4      # .NET SDK kur
        with:
          dotnet-version: '9.0.x'

      - run: dotnet restore                # NuGet paketlerini indir
      - run: dotnet build --no-restore -c Release
      - run: dotnet test --no-build -c Release --logger trx
```

Adımlardan biri bile başarısız olursa kod ana dala giremez. Bozuk kod hiç birikmez.

CD'nin çözümü

CI geçtiyse uygulama otomatik olarak sunucuya kurulur. Veritabanı migration'ları da tam burada çalıştırılır:

```
deploy:
  needs: build-and-test
  runs-on: ubuntu-latest
  steps:
    - name: Migration script üret
      run: dotnet ef migrations script --idempotent --output migrate.sql

    - name: Veritabanını güncelle
      run: sqlcmd -S ${ secrets.DB_SERVER }} -i migrate.sql

    - name: Uygulamayı yayına al
      run: az webapp deploy --src-path ./publish
```

Ara bilgi: Idempotent ne demek?

"Aynı işlemi kaç kez yaparsan yap, sonuç aynı olur" demektir.



Bir asansör düğmesi idempotent'tir: beş kez basmakla bir kez basmak arasında fark yoktur. Ama para çekme işlemi idempotent değildir — beş kez yaparsan beş kat para çıkar.

- **-idempotent** bayrağıyla üretilen SQL script'i her migration için "bu zaten uygulanmış mı?" kontrolü içerir:


```
IF NOT EXISTS (SELECT * FROM [__EFMigrationsHistory] WHERE [MigrationId] =  
N'20260115_AddStock')  
BEGIN  
    ALTER TABLE [Products] ADD [Stock] int NOT NULL DEFAULT 0;  
    INSERT INTO [__EFMigrationsHistory] VALUES (N'20260115_AddStock', N'9.0.0');  
END
```

Script'i yanlışlıkla iki kez çalıştırsan bile hiçbir şey bozulmaz.

Gerçek hayat karşılığı. Lokantada her tabak mutfaktan çıkmadan önce şef kontrol eder: sıcak mı, sunum düzgün mü, siparişle aynı mı? **Bu CI'dir.** Kontrolü geçen tabağın otomatik olarak masaya gitmesi **CD'dir.**

KontROLSÜZ tabak salona çıkmaz. Ve önemlisi: kontrol her tabakta yapılır, ayda bir değil.

Bölüm 3 — Katmanlar ve Aralarındaki İletişim

3.1 Onion Architecture

2008'de Jeffrey Palermo tarafından ortaya atıldı. Klasik katmanlı mimarinin (UI → Business → Data Access) tek bir büyük problemi vardı: **iş kuralları veri erişimine bağımlıydı**. `BusinessLayer` projesi `DataLayer` projesini referans alıyordu. Veritabanını değiştirmek istediğinde iş mantığının da yıkılıyordu.

Onion bunu tersine çevirir. Tek bir kural vardır:

Bağımlılıklar sadece içeriye doğru akar. Dış katman iç katmanı bilir, iç katman dışarıyı asla bilmez.

Katmanlar

Domain (çekirdek). Hiçbir NuGet paketine bağımlı değildir. İdeal olarak `System.*` dışında hiçbir şey yoktur. İçinde ne var:

- **Entity'ler** — kimliği olan, zaman içinde değişen nesneler (`Product` , `Order`)
- **Value Object'ler** — kimliği olmayan, değeriyle tanımlanan nesneler (`Money` , `Address`)
- **Domain Event'ler** — "olan biten" kayıtları (`OrderPlacedEvent`)
- **Arayüzler** — `IProductRepository` , `IEmailService` gibi sözleşmeler
- **Domain istisnaları** — `InsufficientStockException`

Application. Use case'lerin yaşadığı yer. CQRS handler'ları, DTO'lar, validator'lar, pipeline behavior'ları, mapping profilleri. `Domain` 'e bağımlıdır, `Infrastructure` 'a değil.

Infrastructure. Dış dünya. `DbContext` , repository implementasyonları, e-posta servisi, Redis cache, dosya sistemi, üçüncü parti API istemcileri.

Presentation. Web API, gRPC, Blazor, konsol uygulaması. Sadece mesaj gönderir, başka bir şey yapmaz.

Value Object nedir?

Kimliği olmayan, tamamen değeriyle tanımlanan nesne. İki 100 TL birbirinin aynısıdır — "hangi 100 TL" diye sormanın anlamı yoktur. Ama iki `Product` aynı isimde olsa bile farklı ürünlerdir, çünkü ID'leri farklıdır.

```

public sealed record Money
{
    public decimal Amount { get; }
    public string Currency { get; }

    private Money(decimal amount, string currency)
    {
        Amount = amount;
        Currency = currency;
    }

    public static Money Create(decimal amount, string currency)
    {
        if (amount < 0)
            throw new DomainException("Tutar negatif olamaz.");
        if (currency.Length != 3)
            throw new DomainException("Para birimi 3 harfli olmalı.");

        return new Money(amount, currency.ToUpperInvariant());
    }

    public Money Add(Money other)
    {
        if (Currency != other.Currency)
            throw new DomainException("Farklı para birimleri toplanamaz.");
        return new Money(Amount + other.Amount, Currency);
    }
}

```

`decimal Price` yerine `Money Price` kullanmanın kazandırdığı: farklı para birimlerini yanlışlıkla toplamam imkânsız hâle gelir. Bu tür hatalar `decimal` ile çalışırken çalışma zamanında bile fark edilmez.

Rich vs Anemic Domain Model

```

// ANEMIC (kansız) model - sadece veri tutar
public class Product
{
    public int Stock { get; set; } // Herkes istediğini yazabilir
}

// Kural bir servisin içinde, dışarıda
public class ProductService
{
    public void DecreaseStock(Product p, int qty)
    {
        if (p.Stock < qty) throw new Exception("Yetersiz stok");
        p.Stock -= qty;
    }
}

```

Buradaki sorun: `product.Stock = -50;` yazmak hâlâ mümkün. Servisi kullanmayan biri kuralı tamamen atlar.

```
// RICH (zengin) model - veri + davranış bir arada
public class Product : BaseEntity
{
    public string Name { get; private set; }
    public Money Price { get; private set; }
    public int Stock { get; private set; } // private set - dışarıdan yazılamaz
    public Guid CategoryId { get; private set; }

    private Product() { } // EF Core için parametresiz ctor

    public Product(string name, Money price, int stock, Guid categoryId)
    {
        if (string.IsNullOrEmpty(name))
            throw new DomainException("Ürün adı boş olamaz.");
        if (stock < 0)
            throw new DomainException("Başlangıç stoğu negatif olamaz.");

        Name = name;
        Price = price;
        Stock = stock;
        CategoryId = categoryId;

        AddDomainEvent(new ProductCreatedEvent(Id, name));
    }

    public void DecreaseStock(int quantity)
    {
        if (quantity <= 0)
            throw new DomainException("Miktar pozitif olmalı.");
        if (Stock < quantity)
            throw new InsufficientStockException(Id, Stock, quantity);

        Stock -= quantity;
        AddDomainEvent(new StockDecreasedEvent(Id, Stock));
    }
}
```

`private set` sayesinde stok ancak `DecreaseStock` metodundan, kurallar kontrol edilerek değişir. Nesneye nereden erişilirse erişilsin — handler'dan, arka plan görevinden, testten — kural çalışır.

Gerçek hayat karşılığı. Anemic model, tarif kitabı olmayan bir mutfaktır: malzemeler ortada durur, kuralları herkes kafasından uygular. Bir aşçı tavuğu 74 dereceye kadar pişirir, diğeri 60'ta çıkarır.

Rich model ise kuralın malzemenin kendisine yazılı olmasıdır: tavuk paketinin üstünde "74 derecenin altında servis edilemez" yazar ve paket bunu zorlar. Hangi aşçı eline alırsa alsın kural aynıdır.

Dengeli olalım. Anemic model her zaman yanlış değildir. Gerçek bir iş kuralı olmayan CRUD ekranlarında (kategori listesi, ülke tablosu) rich model kurmak boşuna emek harcamaktır. Kural şu: **kural varsa entity'nin içinde olsun; kural yoksa entity'yi zorla zenginleştirme.**

3.2 Dependency Inversion Principle

SOLID prensiplerinin sonuncusu (D). Bu rehberdeki en önemli tek fikir budur.

Önce iki tanım

Bağımlılık (dependency). A sınıfı çalışmak için B sınıfına ihtiyaç duyuyorsa, A B'ye bağımlıdır. B değiştiğinde A da etkilenir.

Soyutlama (abstraction) / arayüz (interface). Ne yapılacağını söyleyen, *nasıl* yapılacağını söylemeyen sözleşme.

```
public interface IEmailService
{
    Task SendAsync(string to, string subject, string body, CancellationToken ct);
}
```

Bu arayüzde tek satır çalışan kod yoktur. Sadece "e-posta gönderen bir şey, şu imzayla bir metoda sahip olmalıdır" der. Gönderimin SMTP ile mi, SendGrid ile mi, güvercinle mi yapıldığı umurunda değildir.

Problem

```
// KÖTÜ
public class OrderService
{
    private readonly SmtEmailService _emailService = new SmtEmailService();
    private readonly SqlServerOrderRepository _repo = new SqlServerOrderRepository();

    public void PlaceOrder(Order order)
    {
        _repo.Save(order);
        _emailService.Send(order.CustomerEmail, "Siparişiniz alındı", "...");
    }
}
```

Dört ayrı problem var:

1. **Ok yanlış yönde.** `OrderService` (iş katmanı), `SmtEmailService` ve `SqlServerOrderRepository`'ye (altyapı) bağımlı. İş kuralı, teknik detaya bağlanmış.
2. **Değişim maliyeti.** SendGrid'e geçmek istersen `OrderService`'i açıp değiştirmen gerekir. Ama `OrderService`'in işi sipariş almaktır, e-posta teknolojisi seçmek değil.

3. **Test edilemez.** Bu sınıfı test ettiğinde gerçekten e-posta gider ve gerçekten SQL Server'a bağlanmaya çalışır. Test yavaş, kırılabilir ve tehlikeli olur.
4. **Karar yanlış yerde.** `new` anahtar kelimesiyle nesneyi kendisi yaratıyor; yani hangi somut sınıfın kullanılacağına kendisi karar veriyor.

Prensibin resmi tanımı

1. Üst seviye modüller alt seviye modüllere bağımlı olmamalıdır. İkisi de soyutlamalara bağımlı olmalıdır.
2. Soyutlamalar detaylara bağımlı olmamalıdır. Detaylar soyutlamalara bağımlı olmalıdır.

Kritik nokta, çoğu anlatımın atladığı şudur: **arayüz iç katmanda tanımlanır, implementasyon dış katmanda.** Arayüzü sadece `interface` yapmak yetmez; onu doğru katmana koymak gerekir.

Çözüm — kodda

```
// ===== Domain katmanı (içerde) – sadece SÖZLEŞME =====
namespace Domain.Repositories;

public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(Guid id, CancellationToken ct);
    Task AddAsync(Order order, CancellationToken ct);
}
```

```
// ===== Application katmanı (ortada) – sözleşmeyi KULLANIR =====
public class PlaceOrderHandler
{
    private readonly IOrderRepository _repo;        // Somut sınıf değil, ARAYÜZ
    private readonly IEmailService _email;
    private readonly IUnitOfWork _unitOfWork;

    // Nesneleri kendisi yaratmıyor, dışarıdan İSTİYOR
    public PlaceOrderHandler(
        IOrderRepository repo, IEmailService email, IUnitOfWork unitOfWork)
    {
        _repo = repo;
        _email = email;
        _unitOfWork = unitOfWork;
    }

    public async Task Handle(PlaceOrderCommand cmd, CancellationToken ct)
    {
        var order = new Order(cmd.CustomerId, cmd.Items);
        await _repo.AddAsync(order, ct);
        await _unitOfWork.SaveChangesAsync(ct);
        await _email.SendAsync(cmd.Email, "Siparişiniz alındı", "...", ct);
    }
}
```

```
// ===== Infrastructure katmanı (dışarda) – sözleşmeyi YERİNE GETİRİR =====
namespace Infrastructure.Persistence.Repositories;

public class EfOrderRepository : IOrderRepository // Domain'deki arayüzü uyguluyor
{
    private readonly AppDbContext _context;
    public EfOrderRepository(AppDbContext context) => _context = context;

    public async Task<Order?> GetByIdAsync(Guid id, CancellationToken ct)
        => await _context.Orders
            .Include(o => o.Lines)
            .FirstOrDefaultAsync(o => o.Id == id, ct);

    public async Task AddAsync(Order order, CancellationToken ct)
        => await _context.Orders.AddAsync(order, ct);
}
```

Artık **Infrastructure** projesi **Domain**'i referans alıyor (arayüzü görebilmek için). **Domain** ise **Infrastructure**'ı görmüyor bile. **Ok tersine döndü.**

Gerçek hayat karşılığı — priz.

Evindeki priz bir arayüzdür. Elektrik santrali prizin şeklini belirlemez; tam tersine, **santral prizin standardına uymak zorundadır**.

Sen buzdolabını takarken elektriğin barajdan mı, rüzgârdan mı, güneşten mi geldiğini bilmezsin ve umursamazsın. Enerji kaynağı tamamen değişse bile evindeki hiçbir cihazı değiştirmen gerekmez.

Standardı kullanan taraf (ev) belirledi, üreten taraf (santral) uydu. Dependency Inversion tam olarak budur — ve **IOrderRepository** 'nin neden **Domain** katmanında durduğunun cevabıdır.

Testte kazandığın şey

```
[Fact]
public async Task PlaceOrder_SiparisiKaydetmeli()
{
    // Sahte (mock) nesneler: gerçek veritabanı yok, gerçek e-posta yok
    var fakeRepo = new Mock<IOrderRepository>();
    var fakeEmail = new Mock<IEmailService>();
    var fakeUow = new Mock<IUnitOfWork>();

    var handler = new PlaceOrderHandler(fakeRepo.Object, fakeEmail.Object, fakeUow.Object);

    await handler.Handle(new PlaceOrderCommand(customerId, items, "a@b.com"), default);

    // "AddAsync tam olarak bir kez çağrıldı mı?"
    fakeRepo.Verify(
        r => r.AddAsync(It.IsAny<Order>(), It.IsAny<CancellationToken>()),
        Times.Once);
}
```

Bu test milisaniyeler içinde çalışır, internet gerektirmez, kimseye e-posta gitmez, veritabanı kurulu olmasa bile geçer. Bu ancak bağımlılıklar arayüz üzerinden **dışarıdan verildiğinde** mümkündür.

Ara bilgi: Mock nedir?

Gerçek nesnenin yerine geçen sahte nesne. Bir arayüzü uygular ama içi boştur; sen ona "şu çağrıldığında şunu döndür" diye tarif edersin. Testte gerçek altyapıyı devre dışı bırakıp sadece test etmek istediğin mantığa odaklanmanı sağlar. .NET'te yaygın kütüphaneler: Moq, NSubstitute, FakeItEasy.

3.3 Dependency Injection (Bağımlılık Enjeksiyonu)

DIP bir **prensiptir** (ne yapılmalı), DI ise onu uygulayan **tekniktir** (nasıl yapılmalı). Sık karıştırılır.

Handler nesneleri kendisi yaratmıyorsa kim yaratıyor? **DI Container** (bağımlılık kabı). Uygulama açılışında ona "hangi arayüz istenirse hangi sınıfı ver" diye tarif edersin:


```
services.AddScoped<IOrderRepository, EfOrderRepository>();
services.AddScoped<IEmailService, SmtEmailService>();
services.AddSingleton<IDateTime, SystemDateTime>();
```

Container bundan sonra **PlaceOrderHandler** yaratması gerektiğinde constructor'ına bakar, **IOrderRepository** istediğini görür, **EfOrderRepository** üretip verir. Buna **constructor injection** denir ve .NET'te varsayılan yöntemdir.



Üç yaşam süresi (lifetime)

Lifetime	Ne zaman yeni nesne üretilir	Ne için uygun
AddSingleton	Bir kez, uygulama boyunca aynı nesne	Cache, konfigürasyon, saat servisi, durumsuz yardımcılar
AddScoped	Her HTTP isteği için bir kez	DbContext, repository, ICurrentUser — en yaygın
AddTransient	Her istendiğinde yeni nesne	Hafif, durum tutmayan küçük servisler

Captive Dependency tuzağı.

Singleton bir servisin içine Scoped bir servis enjekte edemezsin. Singleton uygulama boyunca yaşar; içine koyduğun **DbContext** ilk istek bittiğinde yok edilir. Sonraki isteklerde ölü bir nesneyle çalışmaya çalışırsın.

.NET bunu geliştirme ortamında yakalar ve **Cannot consume scoped service from singleton** hatası verir. Üretim ortamında bu kontrol kapalıysa hata çok daha sinsi şekillerde ortaya çıkar.

3.4 WebAPI'nin referans problemi ve çözümü

Şimdi pratik bir çelişkiye geliyoruz. Bu, çoğu kaynağın üstünden atladığı ama herkesin ilk projesinde takıldığı bir noktadır.

Çelişki

Kural: **WebAPI** sadece **Application**'ı bilmeli. **Infrastructure**'daki **EfOrderRepository** sınıfını görmemeli, çünkü o bir detaydır.

Problem: Ama DI kaydını **Program.cs** içinde yapıyoruz:

```
// WebAPI/Program.cs
services.AddScoped<IOrderRepository, EfOrderRepository>();
//
// ~~~~~
//      Bunu yazabilmek için Infrastructure'ı referans almak zorundayım!
```

Yani kuralı çiğnemedenden uygulamayı ayağa kaldıramıyoruz gibi görünüyor.

Çözüm: Extension method

Önce terimi halledelim.

Extension method (genişletme metodu): Bir sınıfın kaynak koduna dokunmadan ona yeni metod ekleme yöntemi.

`static` bir sınıf içinde, ilk parametresi `this` ile işaretlenmiş `static` bir metod yazarsın:

```
public static class StringExtensions
{
    public static bool IsValidEmail(this string value)
    {
        return !string.IsNullOrEmpty(value)
            && value.Contains('@')
            && value.Contains('.');
    }
}

// Artık her string'in bu metodu varmış gibi kullanabilirsin:
"ali@ornek.com".IsValidEmail(); // true
```

`string` sınıfı Microsoft'a aittir, sen onu değiştiremezsin. Ama extension method ile genişletebilirsin. `this` `string value` yazmanın anlamı: "bu metod string'lere yapışsın, `value` da o string'in kendisi olsun".

Derleyici arka planda `"ali@ornek.com".IsValidEmail()` çağrısını

`StringExtensions.IsValidEmail("ali@ornek.com")` hâline çevirir. Yani sihir değil, söz dizimi kolaylığıdır.

Uygulama: Her katman kendi kaydını kendi yapar

```
// ===== Infrastructure/DependencyInjection.cs =====
// Bu dosya Infrastructure projesinin İÇİNDE.
// Dolayısıyla EfOrderRepository'yi görebiliyor.

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services, IConfiguration configuration)
    {
        services.AddDbContext<AppDbContext>(opt =>
            opt.UseSqlServer(configuration.GetConnectionString("Default")));

        services.AddScoped<IApplicationDbContext>(sp =>
            sp.GetRequiredService<AppDbContext>());
        services.AddScoped<IUnitOfWork>(sp =>
            sp.GetRequiredService<AppDbContext>());

        services.AddScoped<IOrderRepository, EfOrderRepository>();
        services.AddScoped<IProductRepository, EfProductRepository>();
        services.AddScoped<IEmailService, SmtEmailService>();
        services.AddSingleton<IDateTime, SystemDateTime>();

        return services; // Zincirleme yapılabilsin diye kendini döndürüyor
    }
}
```

```
// ===== Application/DependencyInjection.cs =====
public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        var assembly = typeof(DependencyInjection).Assembly;

        services.AddMediatR(cfg =>
        {
            cfg.RegisterServicesFromAssembly(assembly);
            cfg.AddOpenBehavior(typeof(LoggingBehavior<,>));
            cfg.AddOpenBehavior(typeof(ValidationBehavior<,>));
            cfg.AddOpenBehavior(typeof(TransactionBehavior<,>));
        });

        services.AddValidatorsFromAssembly(assembly);
        return services;
    }
}
```

```
// ===== WebAPI/Program.cs =====
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddApplication() // Application kendi kaydını yapar
    .AddInfrastructure(builder.Configuration) // Infrastructure kendi kaydını yapar
    .AddControllers();

var app = builder.Build();
app.UseExceptionHandler();
app.MapControllers();
app.Run();
```

`Program.cs` içinde `EfOrderRepository` kelimesi **hiç geçmiyor**. Sadece "altyapıyı kur" diyor, nasıl kurulduğunu bilmiyor.

Dürüst olalım: kural tam olarak korunuyor mu?

Hayır. `WebAPI` projesi hâlâ teknik olarak `Infrastructure`'ı referans alıyor — yoksa `AddInfrastructure()` metodunu çağıramazdı.

Fark şu: artık **tek bir noktadan, tek bir metotla** temas ediyor. `Infrastructure` içindeki 50 sınıfın hiçbirini tanıtmıyor, hiçbirini `new` ile yaratmıyor, hiçbirinin tipini yazmıyor. Bu tek temas noktasına **composition root** (birleştirme kökü) denir: bağımlılıkların bir araya getirildiği tek yer.

Daha katı olmak isteyenler `WebAPI`'nin `Infrastructure`'ı hiç görmemesi için araya bir `Bootstrapper` projesi koyar. Çoğu ekip için bu aşırıya kaçır ve fayda/maliyet dengesini bozar.

Gerçek hayat karşılığı. Lokantanın müdürü sabah gelir, "mutfak, hazır ol" der ve gider. Hangi tencerenin hangi rafa konulacağını, fırının kaç derecede ısıtılacağını, bulaşık makinesinin hangi programda çalışacağını bilmez ve bilmesine gerek yoktur. Mutfak kendi kurulumunu kendi bilir.

Müdürün mutfaka **tek cümlelik** bir teması vardır. `AddInfrastructure()` = "mutfak, hazır ol".

Bonus: Test için altın değerinde. Bu yapı sayesinde entegrasyon testinde gerçek altyapıyı sahte olanla değiştirebilirsin:

```
builder.Services.AddApplication();
builder.Services.AddTestInfrastructure(); // Gerçek DB yerine Testcontainers veya in-
memory
```

`Application` katmanı hiçbir şeyin değiştiğini fark etmez. Testler gerçek koddan farklı bir yol izlemez.

Bölüm 4 — CQRS ve Handler Dünyası

4.1 CQS'ten CQRS'e

Fikrin atası Bertrand Meyer'in **CQS** (Command Query Separation) prensibidir:

Bir metod ya durumu değiştirir (command), ya veri döndürür (query). İkisini birden yapmaz.

Neden? Çünkü `var x = liste.Pop();` gibi bir çağrı hem sana bir değer verir hem de listeyi değiştirir. Bunu iki kez çağırırsan farklı sonuç alırsın ve bu tür kod hata avında kâbusa dönüşür.

Greg Young bunu mimari seviyeye taşıdı ve **CQRS** (Command Query Responsibility Segregation) oldu: **okuma ve yazma modelleri tamamen ayrı olsun.**

İki tarafın farkları

	Command (yazma)	Query (okuma)
Amaç	Durumu değiştirir	Veri okur
Dönüş değeri	<code>Unit</code> , <code>Guid</code> , <code>Result</code>	DTO veya DTO listesi
Yan etki	Var	Yok
Model	Rich domain entity	Düz DTO, join'li
Veri erişimi	Repository, EF change tracking	<code>AsNoTracking</code> , projection, Dapper
Transaction	Gerekir	Gerekmez
Validasyon	Kapsamlı	Genelde minimal
Ölçekleme	Genelde tek düğüm	Replika, cache, okuma veritabanı

Neden ayırıyoruz?

Çünkü okuma ve yazma ihtiyaçları temelde farklıdır.

Yazarken tutarlılık ve iş kuralı önemlidir. Nesnenin tamamına, tüm ilişkileriyle ihtiyacın olabilir; çünkü kural kontrolü yapacaksın.

Okurken hız ve esneklik önemlidir. Bir ürün listesi ekranı için `Product` entity'sini tüm navigation property'leriyle yüklemek israftır — 4 kolon lazımsa 4 kolon çekersin.

Gerçek hayat karşılığı. Lokantada sipariş almak ile menü göstermek farklı işlerdir.

Sipariş alırken (command) titiz olmak zorundasın: masa numarası doğru mu, alerjen sordun mu, mutfağa doğru gitti mi. Yavaş ama kesin olmalı.

Menü gösterirken (query) hızlı olmalısın. Menüü her müşteri için mutfağa gidip tek tek sormazsın — basılı hâli hazırda durur, hatta duvarda asılıdır. Bir kopyası her masada olabilir.

Yaygın yanlış anlama. CQRS, "iki ayrı veritabanı kullanmak" demek **değildir**. Aynı veritabanını, hatta aynı tabloları kullanabilirsin. Ayrılan şey **modeldir**, veri deposu değil.

İkinci yanlış anlama: CQRS için Event Sourcing zorunlu değildir. İkisi sık birlikte anılır ama bağımsız kavramlardır.

4.2 MediatR

Mediator Pattern nedir?

Birbiriyle konuşması gereken nesnelerin doğrudan konuşmak yerine bir aracı üzerinden haberleşmesi.

Gerçek hayat karşılığı — havaalanı kulesi.

30 uçak havada. Her uçak diğer 29 uçakla ayrı ayrı telsiz kursa kaos olur: $30 \times 29 / 2 = 435$ ayrı bağlantı.

Bunun yerine hepsi sadece kule ile konuşur: 30 bağlantı. Uçaklar birbirini tanımaz, sadece kuleyi tanır. Yeni bir uçak havalandığında diğer 30 uçağın hiçbirine haber vermek gerekmez.

Kule = Mediator.

MediatR olmadan controller

```
// Controller 5 farklı servisi tanıyor
public class ProductsController : ControllerBase
{
    private readonly IProductService _productService;
    private readonly IStockService _stockService;
    private readonly IEmailService _emailService;
    private readonly ILogger<ProductsController> _logger;
    private readonly IValidator<CreateProductDto> _validator;

    public ProductsController(
        IProductService productService, IStockService stockService,
        IEmailService emailService, ILogger<ProductsController> logger,
        IValidator<CreateProductDto> validator)
    {
        _productService = productService;
        _stockService = stockService;
        _emailService = emailService;
        _logger = logger;
        _validator = validator;
    }

    [HttpPost]
    public async Task<IActionResult> Create(CreateProductDto dto)
    {
        var validation = await _validator.ValidateAsync(dto);
        if (!validation.IsValid) return BadRequest(validation.Errors);

        _logger.LogInformation("Ürün oluşturuluyor: {Name}", dto.Name);

        var id = await _productService.CreateAsync(dto);
        await _stockService.InitializeAsync(id, dto.Stock);
        await _emailService.NotifyAdminAsync(id);

        return Ok(id);
    }
}
```

Sorunlar: yeni bir adım eklendiğinde constructor büyür. Aynı iş başka bir yerden (arka plan görevi, konsol aracı) çağrılacaksa bütün bu kodu kopyalarsın. Ve controller, iş akışının detaylarını biliyor — bilmemesi gereken bir şeyi.

MediatR ile

```
// Controller SADECE MediatR'ı tanıyor
public class ProductsController : ControllerBase
{
    private readonly ISender _sender;
    public ProductsController(ISender sender) => _sender = sender;

    [HttpPost]
    public async Task<IActionResult> Create(CreateProductCommand command, CancellationToken
ct)
    {
        var result = await _sender.Send(command, ct);
        return result.IsSuccess ? Ok(result.Value) : BadRequest(result.Error);
    }
}
```

Controller "bu isteği hallet" diyor ve kimin nasıl hallettiğini bilmiyor. Yarın iş akışına 10 yeni adım eklense controller'a hiç dokunulmaz.

ISender mi IMediator mi? IMediator hem Send hem Publish sunar. ISender sadece Send sunar. Controller'da sadece Send kullanacaksan ISender enjekte etmek daha doğrudur — Interface Segregation Principle: bir sınıf kullanmadığı metotlara bağımlı olmamalıdır.

MediatR nasıl çalışıyor?

Sihir yok, üç adım var.



Adım 1 — Açılıştaki tarama. `RegisterServicesFromAssembly` çağrısı, belirtilen assembly'deki tüm tipleri **reflection** ile gezer.

Reflection nedir? Programın çalışırken kendi yapısını inceleyebilmesi. "Bu assembly'de `IRequestHandler<, >` arayüzünü uygulayan hangi sınıflar var?" diye sorabilmesi. Normal kodda tipleri sen yazarsın; reflection'da program tipleri çalışma anında keşfeder.

Bulunan her handler DI container'a **Scoped** olarak kaydedilir.

Adım 2 — Eşleştirme. `Send(command)` çağırıldığında MediatR gelen nesnenin tipine bakar (`CreateProductCommand`), iç sözlüğünden bu tipe karşılık gelen handler'ı bulur (`CreateProductCommandHandler`), DI container'dan onu ister.

Adım 3 — Çağırma. Handler'ın `Handle` metodunu çağırır, sonucu döndürür. Arada varsa pipeline behavior'ları çalıştırır.

Yani MediatR, akıllı bir telefon rehberidir: isim verirsin, numarayı bulur, arar.

Marker interface'ler

```
public interface IRequest<out TResponse> { } // İçi boş!

public interface IRequestHandler<in TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    Task<TResponse> Handle(TRequest request, CancellationToken cancellationToken);
}
```

`IRequest<T>` içi boş bir **marker interface**dir (işaretleyici arayüz). Hiçbir metodu yoktur. Sadece "ben bir mesajım ve `T` tipinde cevap dönerim" der. Amacı davranış eklemek değil, **tip sistemine bilgi vermektir**.

Kural: Bir command'in **tam olarak bir** handler'ı olabilir. İki tane yazarsan MediatR çalışma zamanında hata verir. (Notification'lar için bu kural geçerli değildir — orada çok handler olabilir.)

Lisans notu. MediatR, son sürümlerinde ticari kullanım için ücretli lisansa geçti. Üretim projesi planlıyorsan güncel şartları kontrol et. Alternatifler: Wolverine, Brighter, veya kendi mediator'ını yazmak.

Sonuncusu kulağa zor geliyor ama aslında yaklaşık 60 satırlık bir iştir: DI container'dan handler'ı çözümleyip `Handle` metodunu çağırmaktan ibarettir. Pipeline behavior desteği de eklemek istersen 100 satırı bulur.

4.3 Handler'ın anatomisi

Handler = "İşleyici". Tek bir use case'i baştan sona yürüten sınıf.

Handler'ın tek bir sorumluluğu vardır: **orkestrasyon**. Orkestra şefi gibidir — çalgıcıların ne zaman gireceğini söyler, ama kendisi enstrüman çalmaz.

Handler'ın YAPMADIĞI şeyler



Yapmadığı iş	Kim yapar
Format doğrulaması	Validator
İş kuralı kontrolü	Entity'nin içi
SQL yazmak	Repository / DbContext
Transaction açmak	Pipeline behavior
HTTP durum kodu döndürmek	Controller
Loglama	Pipeline behavior
Cache yönetimi	Pipeline behavior

Bu yüzden sağlıklı bir handler **15-30 satırdır**. 200 satırlık bir handler görürsen, yukarıdaki sorumluluklardan biri kaçmış demektir.

Command handler — tam örnek

```
// 1) Command tanımı
public sealed record CreateProductCommand(
    string Name,
    decimal Price,
    string Currency,
    int Stock,
    Guid CategoryId) : ICommand<Result<Guid>>;
```

```
// 2) Handler
internal sealed class CreateProductCommandHandler
    : IRequestHandler<CreateProductCommand, Result<Guid>>
{
    private readonly IProductRepository _productRepository;
    private readonly ICategoryRepository _categoryRepository;
    private readonly IUnitOfWork _unitOfWork;

    public CreateProductCommandHandler(
        IProductRepository productRepository,
        ICategoryRepository categoryRepository,
        IUnitOfWork unitOfWork)
    {
        _productRepository = productRepository;
        _categoryRepository = categoryRepository;
        _unitOfWork = unitOfWork;
    }

    public async Task<Result<Guid>> Handle(
        CreateProductCommand request, CancellationToken cancellationToken)
    {
        // ADIM 1: Varlık kontrolü (validator yapamaz - DB'ye bakmak gerekir)
        var categoryExists = await _categoryRepository
            .ExistsAsync(request.CategoryId, cancellationToken);

        if (!categoryExists)
            return Result.Failure<Guid>(CategoryErrors.NotFound(request.CategoryId));

        // ADIM 2: Domain nesnesini oluştur
        // İş kuralları Money.Create ve Product ctor'unun İÇİNDE çalışır
        var money = Money.Create(request.Price, request.Currency);
        var product = new Product(request.Name, money, request.Stock, request.CategoryId);

        // ADIM 3: Kaydet
        await _productRepository.AddAsync(product, cancellationToken);
        await _unitOfWork.SaveChangesAsync(cancellationToken);

        return Result.Success(product.Id);
    }
}
```

Üç adım: **kontrol et**, **çalıştır**, **kaydet**. Handler hiçbir iş kuralı yazmıyor — sadece doğru sırayla doğru nesneleri çağırıyor.

İş kuralı neden entity'de?

```
// Product entity'sinin içinde
public void DecreaseStock(int quantity)
{
    if (quantity <= 0)
        throw new DomainException("Miktar pozitif olmalı.");
    if (Stock < quantity)
        throw new InsufficientStockException(Id, Stock, quantity);

    Stock -= quantity;
    AddDomainEvent(new StockDecreasedEvent(Id, Stock));
}
```

Çünkü **Product** nesnesine nereden erişilirse erişilsin kural çalışır: handler'dan, arka plan görevinden, konsol aracından, testten. Kuralı handler'a koysaydın, ikinci bir handler yazan kişi kuralı unutabilirdi — ve bunu fark etmesi aylar sürebilirdi.

internal sealed ne demek?

internal — Bu sınıf sadece kendi assembly'sinin içinden görülebilir. **WebAPI** projesindeki bir geliştirici yanlışlıkla `new CreateProductCommandHandler(...)` yazamaz. Tek giriş kapısı `ISender.Send()`.

sealed — Bu sınıftan kalıtım (inheritance) yapılamaz. Kimse `class MyHandler :` `CreateProductCommandHandler` yazıp davranışı gizlice değiştiremez. Ayrıca derleyiciye küçük bir optimizasyon fırsatı verir (sanal metot çağrılarını doğrudan çağrıya çevirebilir).

Gerçek hayat karşılığı. Mutfaktaki aşçıya salondan kimse doğrudan bağırılmaz — bu **internal** 'dır. Herkes garsona söyler, garson fişi koordinatöre verir, koordinatör aşçıya iletir.

Ve aşçının tarifini kimse gizlice değiştiremez — bu da **sealed** 'dır.

4.4 Query handler ve projection

```
public sealed record GetProductsQuery(
    string? SearchTerm,
    Guid? CategoryId,
    int Page = 1,
    int PageSize = 20) : IQuery<PagedList<ProductListItemDto>>;
```

```

internal sealed class GetProductsQueryHandler
    : IRequestHandler<GetProductsQuery, PagedList<ProductListItemDto>>
{
    private readonly IApplicationDbContext _context;

    public GetProductsQueryHandler(IApplicationDbContext context) => _context = context;

    public async Task<PagedList<ProductListItemDto>> Handle(
        GetProductsQuery request, CancellationToken ct)
    {
        var query = _context.Products.AsNoTracking();

        if (!string.IsNullOrEmpty(request.SearchTerm))
            query = query.Where(p => p.Name.Contains(request.SearchTerm));

        if (request.CategoryId.HasValue)
            query = query.Where(p => p.CategoryId == request.CategoryId.Value);

        // PROJECTION: SELECT * değil, sadece gereken kolonlar
        var projected = query
            .OrderByDescending(p => p.CreatedAt)
            .Select(p => new ProductListItemDto(
                p.Id,
                p.Name,
                p.Price.Amount,
                p.Category.Name,
                p.Stock));

        return await PagedList<ProductListItemDto>
            .CreateAsync(projected, request.Page, request.PageSize, ct);
    }
}

```

Projection neden bu kadar önemli?

`Select(...)` içinde DTO oluşturmaya **projection** denir. EF Core bunu görünce SQL'e sadece o kolonları koyar:

```

SELECT p.Id, p.Name, p.Price, c.Name, p.Stock
FROM Products p
INNER JOIN Categories c ON c.Id = p.CategoryId
WHERE p.IsDeleted = 0
ORDER BY p.CreatedAt DESC
OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY

```

Eğer önce `.ToListAsync()` deyip sonra C# tarafında dönüştürseydin:

```

// KÖTÜ - önce her şeyi çek, sonra ayıkla
var all = await _context.Products.Include(p => p.Category).ToListAsync(ct);
var dtos = all.Select(p => new ProductListItemDto(...)).ToList();

```

Veritabanından **bütün kolonlar ve bütün satırlar** çekilirdi. 100.000 ürünü olan bir tabloda bu, uygulamayı diz çöktürür. Yavaş .NET uygulamalarının en yaygın tek sebebi budur.

Query tarafında repository yok. Dikkat ettiysen query handler doğrudan `IApplicationContext` kullanıyor. Bu bilinçli bir tercihtir.

Repository soyutlaması yazma tarafında değerlidir (aggregate bütünlüğünü korur). Okuma tarafında ise sadece LINQ'in gücünü kısıtlayan bir dolgu katmanı olur: her yeni ekran için repository'ye yeni bir metod eklemek zorunda kalırsın ve repository zamanla 40 metotlu bir çöplüğe döner.

Çok yüksek performans gereken raporlarda Dapper ile ham SQL yazmak da tamamen meşrudur.

4.5 **record**, immutability ve pipeline

Şimdi rehberdeki en incelikli konuya geliyoruz.

Immutable (değişmez) nedir?

Oluşturulduktan sonra içeriği değiştirilemeyen nesne.

```
// MUTABLE (değiştirilebilir)
public class CreateProductCommand
{
    public string Name { get; set; }           // set var -> herkes değiştirebilir
    public decimal Price { get; set; }
}

var cmd = new CreateProductCommand { Name = "Klavye", Price = 450 };
cmd.Price = 0;    // İzin verildi. Ve bu bir felakettir.
```

```
// IMMUTABLE
public sealed record CreateProductCommand(string Name, decimal Price);

var cmd = new CreateProductCommand("Klavye", 450);
cmd.Price = 0;    // Derleme hatası: init-only property
```

record arka planda ne yapıyor?

C# 9 ile gelen **record** anahtar kelimesi, tek satırlık tanımları şuna genişletir:



```
public sealed class CreateProductCommand
{
    public string Name { get; init; } // init = sadece oluşturulurken atanabilir
    public decimal Price { get; init; }

    public CreateProductCommand(string name, decimal price)
    {
        Name = name;
        Price = price;
    }

    // + Equals, GetHashCode, ToString, Deconstruct otomatik üretildi
}
```

record'un bedava verdikleri:

Değer eşitliği. İki **record** içerikleri aynıysa eşittir. Normal **class**'ta iki ayrı nesne asla eşit değildir (referansları farklıdır). Bu, cache anahtarı üretmek için birebir işine yarar.

```
var a = new GetProductQuery(id);
var b = new GetProductQuery(id);
a == b; // record ile: true. class ile: false.
```

Okunabilir ToString(). `CreateProductCommand { Name = Klavye, Price = 450 }` diye basar. Loglamada altın değerindedir. Normal sınıfta sadece tip adını basar.

with ifadesi. Kopyalayıp bir alanı değiştirerek yeni nesne üretir:

```
var indirimli = cmd with { Price = 405 };
// cmd hâlâ 450, indirimli 405. Orijinal bozulmadı.
```

Peki neden bu kadar önemli? Pipeline yüzünden

Pipeline (boru hattı): Bir isteğin, hedefine varmadan önce sırayla geçtiği istasyonlar zinciri.

Bir command şu istasyonlardan geçer:



```

Controller
| Send(command)
v
LoggingBehavior      <- command'i okur, loglar, süre ölçmeye başlar
| next()
v
ValidationBehavior   <- command'i okur, kuralları kontrol eder
| next()
v
TransactionBehavior  <- transaction açar
| next()
v
Handler              <- asıl işi yapar
|
+--- geri dönüş yolu ---
^
TransactionBehavior  <- commit eder
^
ValidationBehavior
^
LoggingBehavior      <- geçen süreyi loglar
^
Controller

```

Aynı `command` nesnesi bu dört istasyondan geçiyor.

Mutable olsaydı ne olurdu?

```

// Hayali kötü senaryo - biri "kampanya indirimi" diye ekledi
public class DiscountBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
{
    public async Task<TResponse> Handle(
        TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        if (request is CreateProductCommand cmd)
            cmd.Price = cmd.Price * 0.9m; // Nesneyi yolda değiştirdi

        return await next();
    }
}

```

Sonuç: `ValidationBehavior` fiyatı 450 olarak doğruladı, ama handler'a 405 gitti. Log'da 450 yazıyor, veritabanında 405 var, arada ne olduğu görünmüyor.

Bu hatayı bulmak günler alır. Immutable olduğunda bu senaryo **derleme zamanında imkânsız** hâle gelir. Doğrulan şey ile işlenen şeyin aynı olduğunu derleyici garanti eder.

Ek fayda — thread güvenliği. Aynı nesneye birden fazla iş parçacığı (thread) aynı anda erişse bile, kimse değiştiremediği için **race condition** (yarış durumu) oluşmaz.

Gerçek hayat karşılığı — sipariş fişi.

Sipariş fişi tükenmez kalemle yazılır ve karbon kopyalıdır. Garson yazar, kasa kontrol eder, koordinatör okur, aşçı uygular. **Kimse üzerine bir şey ekleyip silemez.**

Fiş kurşun kalemle yazılsaydı ve aşçı "3 porsiyon"u "1 porsiyon" yapabilseydi, müşteri şikâyet ettiğinde kimin neyi değiştirdiğini asla bulamazdın.

Değişiklik gerekiyorsa **yeni bir fiş kesilir** (*with* ifadesi), eskisi iptal edilir ve iki fiş de arşivde kalır.

4.6 Pipeline Behavior — detaylı

Bu, MediatR'ın en değerli özelliği ve genelde en az anlaşılan kısmıdır. Behavior'lar matruşka bebek gibi handler'ın etrafını sarar.

```
public interface IPipelineBehavior<in TRequest, TResponse>
{
    Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,    // zincirdeki bir sonraki halka
        CancellationToken cancellationToken);
}
```

Sihir *next* delegesindedir. *await next()* demezen zincir kırılır ve handler **hiç çalışmaz** — buna **kısa devre** (short-circuit) denir. Cache behavior tam olarak bunu kullanır.

Delegate nedir? Bir metodu değişken gibi taşıyabilmeni sağlayan tip.

RequestHandlerDelegate<TResponse>, "çağrıldığında *Task<TResponse>* döndüren bir şey" demektir.

MediatR sana zincirin geri kalanını bu şekilde paketleyip verir.

Validation behavior

```
public sealed class ValidationBehavior<TRequest, TResponse>
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull
{
    private readonly IEnumerable<IValidator<TRequest>> _validators;

    public ValidationBehavior(IEnumerable<IValidator<TRequest>> validators)
        => _validators = validators;

    public async Task<TResponse> Handle(
        TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        if (!_validators.Any())
            return await next(); // Validator yoksa doğrudan geç

        var context = new ValidationContext<TRequest>(request);

        var results = await Task.WhenAll(
            _validators.Select(v => v.ValidateAsync(context, ct)));

        var failures = results
            .SelectMany(r => r.Errors)
            .Where(f => f is not null)
            .ToList();

        if (failures.Count != 0)
            throw new ValidationException(failures); // next() ÇAĞIRILMADI -> handler çalışmaz

        return await next();
    }
}
```

Bu 25 satır sayesinde **hiçbir handler'da validasyon kodu yazmana gerek kalmaz**. Validator'ı yazarsın, behavior otomatik bulur ve uygular.

Transaction behavior

```
public sealed class TransactionBehavior<TRequest, TResponse>
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : ICommand // GENERIC KISIT: sadece command'ler
{
    private readonly AppDbContext _context;

    public TransactionBehavior(AppDbContext context) => _context = context;

    public async Task<TResponse> Handle(
        TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        if (_context.Database.CurrentTransaction is not null)
            return await next(); // İç içe transaction'ı önle

        var strategy = _context.Database.CreateExecutionStrategy();

        return await strategy.ExecuteAsync(async () =>
        {
            await using var transaction = await _context.Database.BeginTransactionAsync(ct);
            try
            {
                var response = await next();
                await transaction.CommitAsync(ct);
                return response;
            }
            catch
            {
                await transaction.RollbackAsync(ct);
                throw;
            }
        });
    }
}
```

Generic kısıt (constraint) mekanizması

`where TRequest : ICommand` satırı çok önemli bir tekniktir. MediatR sadece bu kısıtı sağlayan request'ler için bu behavior'ı devreye sokar. **Query'ler transaction açmaz.**

Bunun için kendi marker interface'lerini tanımlarsın:

```

public interface ICommand : IRequest<Result> { }
public interface ICommand<TResponse> : IRequest<Result<TResponse>> { }

public interface IQuery<TResponse> : IRequest<TResponse> { }

public interface ICachedQuery
{
    string CacheKey { get; }
    TimeSpan Expiration { get; }
}

```

Artık `CachingBehavior<,>` yazdığında `where TRequest : ICachedQuery` diyebilirsin ve sadece cache'lenmek isteyen query'ler o behavior'a girer. Diğerleri hiç uğramaz.

Kayıt sırası kritiktir

```

services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssembly(typeof(ApplicationAssembly).Assembly);

    cfg.AddOpenBehavior(typeof(LoggingBehavior<,>)); // 1. EN DIŞ
    cfg.AddOpenBehavior(typeof(ValidationBehavior<,>)); // 2.
    cfg.AddOpenBehavior(typeof(CachingBehavior<,>)); // 3.
    cfg.AddOpenBehavior(typeof(TransactionBehavior<,>)); // 4. EN İÇ, handler'a en yakın
});

```

Behavior'lar kaydedildikleri sırayla dıştan içe dizilir. Bu sıralamanın mantığı:

- **Logging en dışta** — her şeyi, hata dâhil, görebilsin. Toplam süreyi ölçebilsin.
- **Validation transaction'dan önce** — geçersiz istek için boşuna transaction açmayalım.
- **Caching validation'dan sonra** — geçersiz istekleri cache'lemeyelim.
- **Transaction en içte** — transaction ne kadar kısa süre açık kalırsa o kadar iyidir. Uzun süre açık kalan transaction, veritabanında kilit tutar ve diğer işlemleri bekletir.

4.7 Notification (Domain Event) — bire çok

`IRequest` **bire birdir**: bir command'ın tek handler'ı olur.

`INotification` **bire çoktur**: bir olayın sıfır veya daha fazla handler'ı olabilir.

```

public sealed record ProductCreatedEvent(Guid ProductId, string Name) : INotification;

```

```
// Handler 1
internal sealed class SendNotificationOnProductCreated
    : INotificationHandler<ProductCreatedEvent>
{
    private readonly IEmailService _emailService;

    public Task Handle(ProductCreatedEvent e, CancellationTokens ct)
        => _emailService.SendAsync("admin@ornek.com", $"Yeni ürün: {e.Name}", "...", ct);
}

// Handler 2 - aynı olay, tamamen farklı iş
internal sealed class InvalidateCacheOnProductCreated
    : INotificationHandler<ProductCreatedEvent>
{
    private readonly ICacheService _cache;

    public Task Handle(ProductCreatedEvent e, CancellationTokens ct)
        => _cache.RemoveByPrefixAsync("products:", ct);
}

// Handler 3 - arama motoruna ekle
internal sealed class IndexProductOnCreated
    : INotificationHandler<ProductCreatedEvent>
{
    public Task Handle(ProductCreatedEvent e, CancellationTokens ct)
        => _searchIndex.AddAsync(e.ProductId, e.Name, ct);
}
```

Üçü de çalışır. **Command tarafı bu handler'ların varlığından habersizdir.** Yeni bir davranış eklemek için mevcut kodu değiştirmezsin, yeni bir handler dosyası eklersin. Bu, SOLID'in **O**'sudur: Open/Closed Principle — genişlemeye açık, değişikliğe kapalı.

Gerçek hayat karşılığı — anons sistemi.

Lokantada "12 numaralı masa siparişi hazır" diye anons edilir. Bu anonsu garson duyar (tabağı alır), bulaşıkçı duyar (tencereyi hazırlar), müdür duyar (istatistiğe işler).

Anonsu yapan kişi kimin dinlediğini bilmez. Yarın bir kişi daha dinlemeye başlasa anonsu yapan hiçbir şey değiştirmez.

Dikkat — Publish'in tuzağı. Publish varsayılan olarak handler'ları **sırayla** çalıştırır ve **biri hata verirse diğerleri hiç çalışmaz.**

Yukarıdaki örnekte e-posta gönderimi başarısız olursa cache temizlenmez ve arama motoru güncellenmez. Üstelik veritabanı işlemi çoktan tamamlanmış olabilir — yani sistem tutarsız bir durumda kalır.

Çözüm: Outbox Pattern. Event'i, ana işlemle **aynı transaction içinde** bir **OutboxMessages** tablosuna yazarsın. Aynı bir arka plan görevi bu tabloyu okuyup event'leri işler, başarısız olanları tekrar dener.

Böylece "veritabanına kaydedildi ama e-posta gitmedi" ya da tersi durumu ortadan kaldır. Detayı Bölüm 6'da.



Bölüm 5 — Bir İsteğin Uçtan Uca Yolculuğu

Şimdiye kadar öğrendiğimiz her şeyi tek bir senaryoda birleştirelim: **bir müşteri sipariş veriyor.**

Adım adım

1. Müşteri isteği gönderir.

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{ "customerId": "a3f2...", "items": [ { "productId": "b7c1...", "quantity": 2 } ] }
```

2. Model binding. ASP.NET Core JSON'u **deserialize** edip **PlaceOrderCommand** nesnesine dönüştürür. Bu nesne bir **record**'dur, yani **immutable**'dur.

3. Controller devreye girer. Tek satır kod içerir:

```
var result = await _sender.Send(command, ct);
```

Controller ne olacağını bilmez. Sadece mesajı iletir.

4. MediatR handler'ı bulur. Reflection ile açılıştan oluşturulmuş sözlükten **PlaceOrderCommandHandler**'ı bulur. Ama önce pipeline'a girer.

5. LoggingBehavior. Kronometreyi başlatır, isteği loglar. **record**'un **ToString()**'i sayesinde log okunabilir.

6. ValidationBehavior. **PlaceOrderCommandValidator**'ı çalıştırır: müşteri ID'si boş mu, en az bir kalem var mı, miktarlar pozitif mi? Hata varsa **next()** çağrılmaz ve **handler hiç çalışmaz.**

7. TransactionBehavior. **where TRequest : ICommand** kısıtını sağladığı için devreye girer, transaction açar.

8. Handler çalışır. Üç adım: kontrol et, çalıştır, kaydet.

```

public async Task<Result<Guid>> Handle(PlaceOrderCommand request, CancellationToken ct)
{
    // Varlık kontrolü - validator yapamazdı, DB'ye bakmak gerekti
    var customer = await _customerRepository.GetByIdAsync(request.CustomerId, ct);
    if (customer is null)
        return Result.Failure<Guid>(CustomerErrors.NotFound(request.CustomerId));

    var order = new Order(customer.Id);

    foreach (var item in request.Items)
    {
        var product = await _productRepository.GetByIdAsync(item.ProductId, ct);
        if (product is null)
            return Result.Failure<Guid>(ProductErrors.NotFound(item.ProductId));

        product.DecreaseStock(item.Quantity); // İŞ KURALI entity'nin içinde
        order.AddLine(product.Id, item.Quantity, product.Price);
    }

    await _orderRepository.AddAsync(order, ct);
    await _unitOfWork.SaveChangesAsync(ct);

    return Result.Success(order.Id);
}

```

9. DIP burada devreye girer. Handler `IProductRepository` arayüzünü kullanıyor. Arkasında `EfProductRepository`, onun da arkasında `DbContext` ve SQL Server var — ama handler bunların hiçbirini bilmiyor. Testte bu arayüzün yerine bir mock koyabilirsin.

10. Entity kuralı uygular. `product.DecreaseStock(2)` çağrısı stok yetersizse istisna fırlatır. Bu kural handler'da değil, `Product` sınıfının içindedir — dolayısıyla nereden çağrılırsa çağrılсын çalışır.

11. DbContext SQL üretir. `SaveChangesAsync()` change tracker'ı okur, hangi nesnelerin değiştiğini bulur, Fluent API ile tanımlanmış şemaya göre SQL üretir. Aynı anda `SaveChanges` override'ı `CreatedAt` ve `CreatedBy` alanlarını doldurur.

12. Domain event'ler yayınlanır. Veritabanı işlemi başarılı olduktan sonra `OrderPlacedEvent` yayınlanır. Üç ayrı notification handler tetiklenir: müşteriye e-posta, stok uyarısı kontrolü, satış istatistiği güncellemesi.

13. TransactionBehavior commit eder.

14. LoggingBehavior süreyi yazar. "PlaceOrderCommand tamamlandı: 47ms".

15. Controller sonucu HTTP'ye çevirir.

```

return result.IsSuccess
    ? CreatedAtAction(nameof(GetById), new { id = result.Value }, result.Value)
    : BadRequest(result.Error);

```

16. Sonuç DTO'ya maplenir, JSON'a serialize edilir, müşteriye döner.

17. Ve tüm bu kod, **CI/CD** hattından testleri geçerek sunucuya çıkmıştır.

Dikkat et: Bu 17 adımda handler sadece 8. adımdaydı ve 20 satır kod içeriyordu. Geri kalan her şey — loglama, doğrulama, transaction, audit alanları, event yayını, hata dönüşümü — **başka birinin işiydi ve otomatik oldu.**

İyi mimarinin ölçüsü budur: yeni bir use case yazdığında ne kadar az şey düşünmen gerektiği.

Bölüm 6 — Design Pattern Sözlüğü

6.1 Repository Pattern

Veri erişimini bir bellek içi koleksiyon gibi soyutlar.

Generic Repository neden çoğu zaman anti-pattern?

```
// KAÇIN
public interface IRepository<T> where T : class
{
    IQueryable<T> GetAll(); // IQueryable sızdırmak soyutlamayı anlamsız kılar
    Task<T> GetByIdAsync(int id);
    void Add(T entity);
    void Update(T entity);
    void Delete(T entity);
}
```

Problemler:

1. `DbContext` zaten Repository + Unit of Work'tür. Üstüne bir jenerik katman koymak sadece bir dolgu ekler.
2. `IQueryable` döndürmek soyutlamayı yok eder — çağırın taraf zaten EF Core'a özgü sorgu yazacaktır.
3. EF Core'un `Include`, `AsSplitQuery`, `ExecuteUpdate`, `HasQueryFilter` gibi yeteneklerini gizler.
4. Her entity için aynı beş metod, ama gerçekte her entity'nin ihtiyacı farklıdır.

Doğru kullanım: Aggregate bazlı, spesifik repository

```
public interface IOrderRepository
{
    Task<Order?> GetWithLinesAsync(Guid id, CancellationToken ct);
    Task<IReadOnlyList<Order>> GetPendingOlderThanAsync(DateTime date, CancellationToken ct);
    Task AddAsync(Order order, CancellationToken ct);
}
```

Her metodun bir amacı vardır ve isminden ne yaptığı anlaşılır.

Aggregate nedir? Birlikte değişen ve birlikte tutarlı olması gereken entity grubu. `Order` ve `OrderLine` bir aggregate'tir: sipariş satırı tek başına anlamsızdır, hep siparişle birlikte yüklenir ve kaydedilir. Gruptaki "baş" entity'ye **aggregate root** denir. Repository'ler aggregate root başına açılır — `OrderLineRepository` yazmazsın.

6.2 Unit of Work

Birden fazla değişikliği tek bir atomik işlemde toplar.

```
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(CancellationToken ct = default);
}

// ApplicationDbContext : DbContext, IApplicationDbContext, IUnitOfWork
```

Kritik kural: **repository'ler SaveChanges çağırmaz**. Sadece **Add / Update** yapar. Kaydetme sorumluluğu handler'da veya bir **UnitOfWorkBehavior**'dadır.

Neden? Çünkü bir handler içinde üç farklı repository'ye dokunup hepsini **tek transaction'da** kaydetmek istersin. Her repository kendi başına kaydetseydi, ikincisi hata verdiğinde birincisi çoktan yazılmış olurdu.

Gerçek hayat karşılığı. Alışveriş sepeti. Ürünleri tek tek sepete atarsın ama para tek seferde ödenir. Kasada bir sorun çıkarsa hiçbirini satın alınmamış olur — yarısı senin, yarısı mağazanın değil.

6.3 Specification Pattern

Sorgu kriterlerini nesneleştirir; tekrar kullanılabilir ve test edilebilir hâle getirir.

```
public sealed class ActiveProductsInCategorySpec : Specification<Product>
{
    public ActiveProductsInCategorySpec(Guid categoryId)
    {
        Criteria = p => p.CategoryId == categoryId && !p.IsDeleted && p.Stock > 0;
        AddInclude(p => p.Category);
        ApplyOrderBy(p => p.Name);
    }
}

// Kullanımı
var products = await _repository.ListAsync(new ActiveProductsInCategorySpec(id), ct);
```

Aynı "aktif ürün" tanımını beş yerde kopyalamak yerine tek yerde tanımlarsın. Tanım değişince tek yer değişir. Ayrıca spec'i tek başına birim testine sokabilirsin.

Hazır kütüphaneler: Ardalis.Specification.

6.4 Result Pattern

Beklenen hataları exception ile yönetmek pahalıdır (yığın çözme / stack unwinding) ve akışı gizler. Onun yerine sonucu tip olarak döndürürsün.

```

public class Result
{
    public bool IsSuccess { get; }
    public bool IsFailure => !IsSuccess;
    public Error Error { get; }

    protected Result(bool isSuccess, Error error)
    {
        IsSuccess = isSuccess;
        Error = error;
    }

    public static Result Success() => new(true, Error.None);
    public static Result Failure(Error error) => new(false, error);
    public static Result<T> Success<T>(T value) => new(value, true, Error.None);
    public static Result<T> Failure<T>(Error error) => new(default, false, error);
}

public sealed record Error(string Code, string Message)
{
    public static readonly Error None = new(string.Empty, string.Empty);
}

```

Hata tanımlarını merkezî bir yerde toplamak, hem tutarlılık hem de çeviri kolaylığı sağlar:

```

public static class ProductErrors
{
    public static Error NotFound(Guid id) =>
        new("Product.NotFound", $"{id} kimlikli ürün bulunamadı.");

    public static readonly Error InsufficientStock =
        new("Product.InsufficientStock", "Yetersiz stok.");
}

```

Kural: Beklenen durumlar için `Result`, gerçekten istisnai durumlar için `Exception`.

"Ürün bulunamadı" beklenen bir durumdur — kullanıcı yanlış ID girmiş olabilir, bu normaldir. "Veritabanı bağlantısı koptu" istisnaidir — bu olmamalıydı.

Ölçüt şudur: **bu durum akışın normal bir parçası mı, yoksa bir arıza mı?**

6.5 Diğer desenler

Factory

Karmaşık nesne yaratımını gizler ve anlamlı isim verir.

```
// new Order(...) yerine:
var order = Order.CreateFromCart(customer, cart, discountPolicy);
```

Static factory metodun avantajı: `new` ile aynı isimde birden fazla ctor yazamazsın ama farklı isimlerde factory metotları yazabilirsin. `Order.CreateDraft()`, `Order.CreateFromQuote()` gibi.

Strategy

Aynı işi farklı algoritmalarla yapma.

```
public interface IDiscountStrategy
{
    Money Calculate(Order order);
}

public sealed class PercentageDiscount : IDiscountStrategy { }
public sealed class FixedAmountDiscount : IDiscountStrategy { }
public sealed class BuyOneGetOneFree : IDiscountStrategy { }
```

Uzun `if/else if/else` zincirlerini öldürür. Yeni bir indirim türü eklemek için mevcut kodu değil, yeni bir sınıf eklersin.

Decorator

Mevcut bir implementasyonu değiştirmeden sarmalayıp davranış ekler.

```
public sealed class CachedProductRepository : IProductRepository
{
    private readonly IProductRepository _inner; // Gerçek repository
    private readonly ICacheService _cache;

    public async Task<Product?> GetByIdAsync(Guid id, CancellationToken ct)
    {
        var cached = await _cache.GetAsync<Product>($"product:{id}", ct);
        if (cached is not null) return cached;

        var product = await _inner.GetByIdAsync(id, ct); // Gerçek olana devret
        if (product is not null)
            await _cache.SetAsync($"product:{id}", product, ct);

        return product;
    }
}
```

Kayıt Scrutor kütüphanesiyle tek satırdır:

```
services.AddScoped<IProductRepository, EfProductRepository>();
services.Decorate<IProductRepository, CachedProductRepository>();
```

`EfProductRepository` cache'in varlığından haberdar değildir. Cache'i kaldırmak istersen tek satır silersin.

Adapter

Uyumsuz iki arayüzü birbirine bağlar. Üçüncü parti bir ödeme SDK'sını kendi `IPaymentGateway` arayüzüne uydurmak gibi. Sağlayıcı değiştiğinde sadece adapter değişir, uygulamanın geri kalanı aynı kalır.

Options Pattern

Konfigürasyonu tip güvenli hâle getirir.

```
public sealed class JwtSettings
{
    public string Issuer { get; init; } = string.Empty;
    public string Audience { get; init; } = string.Empty;
    public int ExpiryMinutes { get; init; }
}

// Kayıt
services.Configure<JwtSettings>(configuration.GetSection("Jwt"));

// Kullanım
public class TokenService(IOptions<JwtSettings> options)
{
    private readonly JwtSettings _settings = options.Value;
}
```

Üç varyant: `IOptions<T>` (singleton, sabit), `IOptionsSnapshot<T>` (scoped, her istekte yeniden okunur), `IOptionsMonitor<T>` (singleton, değişiklik bildirimi alır).

Outbox Pattern

Dağıtık sistemlerde "veritabanına yaz + mesaj kuyruğa at" işleminin atomikliğini garanti eder.

Problem: veritabanına yazdın, sonra mesaj kuyruğuna yazarken uygulama çöktü. Veri var ama kimse haberdar değil. Ya da tersi: mesajı gönderdin, veritabanı işlemi geri alındı. Herkes olmayan bir şeyi işliyor.

Çözüm: mesajı **aynı transaction içinde** bir tabloya yaz.

```
// SaveChanges sırasında, aynı transaction içinde
_context.OutboxMessages.Add(new OutboxMessage
{
    Id = Guid.NewGuid(),
    Type = nameof(OrderPlacedEvent),
    Content = JsonSerializer.Serialize(orderPlacedEvent),
    OccurredOn = DateTime.UtcNow,
    ProcessedOn = null
});
```

Ayrı bir arka plan görevi (Quartz, Hangfire veya **BackgroundService**) bu tabloyu okur, mesajları işler, **ProcessedOn** alanını doldurur. Başarısız olanları tekrar dener.

Saga / Process Manager

Mikroservislerde uzun süreli iş akışlarını yönetir. Dağıtık transaction yerine **telafi edici işlemler** (compensating transactions) kullanır.

Örnek akış: sipariş oluştur → ödeme al → stok düş → kargo oluştur. Ödeme başarısız olursa siparişi iptal et. Kargo oluşturulamazsa ödemeyi iade et ve stoğu geri ver.

Event Sourcing

Durumun kendisini değil, ona götüren tüm event'leri saklarsın. Mevcut durum, event'lerin baştan oynatılmasıyla (replay) hesaplanır.

Kazandırdığı: Tam denetim izi, zamanda geri gitme, "bu kayıt nasıl bu hâle geldi" sorusuna kesin cevap.

Bedeli: Ciddi karmaşıklık, şema evrimi problemi (5 yıl önceki event formatını hâlâ okuyabilmen gerekir), sorgulama zorluğu (anlık durum için ayrı bir projeksiyon tutmak zorundasın).

Event Sourcing, CQRS'in zorunlu parçası değildir. Bu, alandaki en yaygın yanlış anlamalardan biridir. CQRS'i Event Sourcing olmadan kullanabilirsin ve çoğu proje tam olarak bunu yapar.

Bölüm 7 — Mimari Yaklaşımlar

7.1 N-Tier / Layered

Klasik UI → BLL (Business Logic Layer) → DAL (Data Access Layer). Basit, herkesin bildiği, küçük projeler için yeterli.

Sorunu: Bağımlılık yönü yukarıdan aşağıdır. İş mantığı veri erişimine bağlanır. Veritabanını değiştirmek iş kurallarını etkiler.

7.2 Hexagonal (Ports & Adapters)

Alistair Cockburn, 2005. Onion'un atası.

Merkezde iş mantığı vardır. Etrafında **port**'lar (arayüzler) ve bu portlara takılan **adapter**'lar (implementasyonlar) bulunur.

- **Driving adapter** — uygulamayı tetikleyen taraf: REST controller, CLI, zamanlanmış görev
- **Driven adapter** — uygulama tarafından tetiklenen taraf: repository, e-posta servisi, mesaj kuyruğu

Felsefesi Onion ile aynıdır; Onion katmanları daha net isimlendirir.

7.3 Clean Architecture

Robert C. Martin, 2012. Onion + Hexagonal + Screaming Architecture sentezi.

Katmanları Entities / Use Cases / Interface Adapters / Frameworks & Drivers olarak adlandırır. Pratikte Onion ile arasındaki fark isimlendirmeden ibarettir; .NET dünyasında ikisi çoğunlukla aynı şeyi kastetmek için kullanılır.

7.4 Vertical Slice Architecture

Jimmy Bogard. Son yıllarda Onion'a ciddi bir alternatif olarak yükseldi.

Yatay katmanlar yerine **dikey özellik dilimleri**. Klasörleme teknik role göre değil, işe göre yapılır:

```
Features/
  Products/
    CreateProduct/      -> Command + Handler + Validator + Endpoint TEK dosyada
    GetProducts/
    DecreaseStock/
  Orders/
    PlaceOrder/
    CancelOrder/
```

Mantığı: Bir özelliği değiştirirken 6 farklı klasörde dolaşmak yerine tek klasörde çalışırsın. Katmanlar arası "gereksiz soyutlama vergisi" ödemezsin — basit bir CRUD dilimi doğrudan **DbContext** kullanabilirken, karmaşık bir dilim tam DDD uygulayabilir.

Kazandırdığı: Dilimler birbirinden bağımsızdır, bu yüzden bir dilimi değiştirmek diğerlerini bozmaz. Her dilim kendi karmaşıklık seviyesini seçer.

Bedeli: Kod tekrarı artar (aynı sorgu iki dilimde ayrı yazılabilir). Disiplin gerektirir; aksi hâlde dilimler arası tutarlılık kaybolur.

CQRS ile mükemmel uyuşur — zaten her dilim bir command veya query'dir.

7.5 Modular Monolith

Tek deployment birimi, ama içeride sıkı sınırlanmış modüller. Modüller birbirine sadece public contract'lar üzerinden erişir; veritabanı şemaları ayrıdır.

Mikroservislerin dağıtık sistem acısını çekmeden sınır disiplini kazandırır. **Çoğu ekip için doğru başlangıç noktasıdır** — gerektiğinde bir modülü çıkarıp servise dönüştürsün.

7.6 Microservices

Her servis kendi veritabanı, kendi deployment'ı.

Gerçek ihtiyaç: Bağımsız ölçeklenme ve bağımsız ekip hızı. 200 kişilik bir organizasyonda 15 ekibin birbirini beklemeden yayına çıkabilmesi.

Bedeli: Dağıtık transaction yokluğu, eventual consistency, servis keşfi, dağıtık izleme (tracing), ağ gecikmesi, veri tutarlılığı problemleri, çok daha karmaşık test.

Ekip 10 kişiden azsa mikroservis neredeyse her zaman erken bir karardır. Martin Fowler'ın deyişiyle: mikroservise geçmenin ön koşulu, monolit'i düzgün modüllere bölebilmiş olmaktır. Bunu yapamayan ekip, dağıtık bir çamur yumağı üretir.

Bölüm 8 — Proje İskeleti

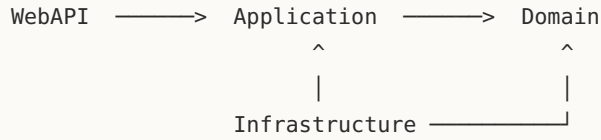
```

src/
├── Domain/                                <- Hiçbir dış bağımlılık
│   ├── Common/                          BaseEntity, ValueObject, IDomainEvent, Result, Error
│   ├── Entities/                        Product, Order, Category
│   ├── ValueObjects/                    Money, Address, Email
│   ├── Events/                          ProductCreatedEvent, OrderPlacedEvent
│   ├── Exceptions/                      DomainException, InsufficientStockException
│   └── Repositories/                    IProductRepository, IOrderRepository (SADECE arayüz)
├── Application/                          <- Domain'e bağımlı
│   ├── Common/
│   │   ├── Behaviors/                   Validation, Logging, Transaction, Caching, Performance
│   │   ├── Interfaces/                  IApplicationDbContext, IEmailService, ICurrentUser, IDateTime
│   │   ├── Messaging/                   ICommand, IQuery, ICachedQuery (marker interface'ler)
│   │   ├── Mappings/                     Mapster / AutoMapper profilleri
│   │   └── Models/                       PagedList<T>, Result<T>
│   ├── Features/
│   │   ├── Products/
│   │   │   ├── Commands/                CreateProduct/, UpdateProduct/, DeleteProduct/
│   │   │   └── Queries/                  GetProducts/, GetProductById/
│   │   └── Orders/
│   │       ├── Commands/                PlaceOrder/, CancelOrder/
│   │       └── Queries/                  GetOrderHistory/
│   └── DependencyInjection.cs
├── Infrastructure/                        <- Application ve Domain'e bağımlı
│   ├── Persistence/
│   │   ├── AppDbContext.cs
│   │   ├── Configurations/              ProductConfiguration (IEntityTypeConfiguration)
│   │   ├── Repositories/                 EfProductRepository, EfOrderRepository
│   │   ├── Interceptors/                 AuditableEntityInterceptor
│   │   └── Migrations/
│   ├── Services/                         SmtplibEmailService, RedisCacheService, JwtTokenService
│   ├── Outbox/                           OutboxMessage, OutboxProcessorJob
│   └── DependencyInjection.cs
└── WebAPI/                               <- Application'a bağımlı
    ├── Controllers/                       (veya Endpoints/ - Minimal API tercih edilirse)
    ├── Middleware/                         GlobalExceptionHandler
    ├── Program.cs
    └── appsettings.json

tests/
├── Domain.UnitTests/                      Entity iş kuralları - mock GEREKMEZ, en hızlı testler
├── Application.UnitTests/                  Handler'lar - repository'ler mock'lanır
├── Application.IntegrationTests/           Testcontainers ile gerçek SQL Server
└── Architecture.Tests/                     Katman kurallarını doğrulayan testler

```

Referans yönleri



Domain hiç kimseyi referans almaz. Bu, iyi niyete bırakılacak bir şey değildir — otomatik testle garanti altına alınmalıdır.

Mimari testler

NetArchTest.Rules kütüphanesiyle katman kurallarını test hâline getirebilirsin:

```

[Fact]
public void Domain_HicbirDiskatmanaBagimliOlmamali()
{
    var result = Types.InAssembly(typeof(Product).Assembly)
        .Should()
        .NotHaveDependencyOnAny("Application", "Infrastructure", "WebAPI")
        .GetResult();

    result.IsSuccessful.Should().BeTrue(
        $"Şu tipler kuralı çiğniyor: {string.Join(", ", result.FailingTypeNames ?? [])}");
}

[Fact]
public void Handlerlar_Internal_Ve_Sealed_Olmali()
{
    var result = Types.InAssembly(typeof(CreateProductCommandHandler).Assembly)
        .That().ImplementInterface(typeof(IRequestHandler<, >))
        .Should().NotBePublic()
        .And().BeSealed()
        .GetResult();

    result.IsSuccessful.Should().BeTrue();
}
  
```

Bu testler CI hattında çalışır. Biri kuralı çiğnediğinde pull request geçmez. **Mimari, dokümandaki bir öneri olmaktan çıkıp derleyici tarafından zorlanan bir kurala dönüşür.**

Bölüm 9 — Ne Zaman Ne Kullanmalı?

Bu rehberdeki her yapı bir maliyet karşılığında gelir: daha fazla dosya, daha uzun onboarding süresi, hata ayıklarken daha fazla "bu çağrı nereye gidiyor" sorusu.

Karar tablosu

Durum	Öneri
3 tablolü admin paneli	Tek proje, doğrudan <code>DbContext</code> , servis sınıfı yok
Küçük ama büyüyecek proje	Vertical Slice + MediatR, katman ayrımı hafif
Orta ölçekli iş uygulaması	Onion + CQRS + spesifik repository'ler
Karmaşık domain, çok kural	Onion + DDD + rich model + domain event'ler
Çok ekipli büyük sistem	Modular Monolith, gerekirse mikroservis
Yüksek okuma trafiği	CQRS'te okuma tarafını ayır: Dapper, cache, replika
Denetim izi zorunlu	Outbox + audit log; gerekirse Event Sourcing

Sağlıklı ilerleme sırası

1. **Sade başla.** Katmanları ayır ama içlerini basit tut. Repository yazma, doğrudan `DbContext` kullan.
2. **Acıyı bekle.** Bir şey gerçekten canını yaktığında soyutlama ekle. "İleride lazım olur" diye ekleme.
3. **Sınırları baştan koy.** `Domain`'in temiz kalması gibi kuralları en baştan uygula — bunları sonradan düzeltmek çok pahalıdır.
4. **Testle zorla.** Mimari kuralları mimari testlerle garanti altına al.

Ayrım şudur: Karmaşıklığı ihtiyaç ortaya çıkmadan **eklemeyin**. Ama sınırları en baştan **koyun**.

Soyutlama eklemek sonradan kolaydır — 50 dosyayı dolaşırsın, biraz uğraşırsın, biter. `Domain` katmanına sızmış EF Core bağımlılıklarını sonradan sökmek ise projeyi baştan yazmakla eş değerdir.

Ek A — Terimler Sözlüğü

Terim	Kısa açıklama
Aggregate	Birlikte değişen, birlikte tutarlı olması gereken entity grubu
Aggregate Root	Aggregate'e dışarıdan erişimin tek kapısı olan entity
Anemic Model	Sadece veri tutan, davranışı olmayan entity
Anonim tip	Adı olmayan, anlık oluşturulan nesne: <code>new { A = 1, B = 2 }</code>
API	Programın dışarıya açtığı kapı, sözleşme
Assembly	Derlenmiş .NET projesi; <code>.dll</code> veya <code>.exe</code>
Attribute	Sınıf/metot üstüne yapıştırılan etiket: <code>[HttpGet]</code>
Behavior	Handler'ın etrafını saran pipeline halkası
Change Tracker	EF Core'un nesnelerdeki değişiklikleri izleyen mekanizması
CLR	.NET kodunu çalıştıran motor
Command	Durumu değiştiren mesaj
Composite Key	Birden fazla kolonun birlikte oluşturduğu birincil anahtar
Composition Root	Bağımlılıkların bir araya getirildiği tek nokta
CQRS	Okuma ve yazma modellerinin ayrılması
CRUD	Create, Read, Update, Delete
Delegate	Metodu değişken gibi taşımayı sağlayan tip
Deserialization	JSON metnini nesneye çevirme
DI Container	Bağımlılıkları çözümleyip enjekte eden altyapı
DIP	Bağımlılığın tersine çevrilmesi prensibi
Domain Event	Domain'de olan bir şeyin kaydı: <code>OrderPlacedEvent</code>
DTO	Sadece veri taşıyan, davranışsız nesne
Entity	Kimliği olan, zaman içinde değişen nesne
Extension Method	Sınıfın koduna dokunmadan metot ekleme tekniği
Fluent API	Zincirlenmiş metotlarla konfigürasyon
Handler	Tek bir use case'i yürüten sınıf
Idempotent	Kaç kez tekrarlanırsa tekrarlınsın sonucu aynı olan işlem

Terim	Kısa açıklama
Immutable	Oluşturulduktan sonra değiştirilemeyen nesne
internal	Sadece kendi assembly'sinden görülebilen
Lambda	İsimsiz, kısa fonksiyon: <code>x => x.Name</code>
Marker Interface	İçi boş, sadece işaretleme amaçlı arayüz
Mediator	Nesneler arası iletişimi merkezileştiren aracı
Migration	Veritabanı şema değişikliğini tarif eden dosya
Mock	Testte gerçek nesnenin yerine geçen sahte nesne
Model Binding	HTTP isteğini C# nesnesine dönüştürme
Notification	Birden çok handler'ı olabilen olay mesajı
ORM	Nesne ile veritabanı arasında tercümanlık yapan kütüphane
Outbox	Mesaj gönderimini veritabanı transaction'ıyla atomik yapan desen
Pipeline	İsteğin geçtiği istasyonlar zinciri
POCO	Hiçbir kütüphaneye bağımlı olmayan sade C# sınıfı
Projection	Sorguda sadece gereken kolonları seçme
Query	Veri okuyan, durumu değiştirmeyen mesaj
Race Condition	Aynı veriye eşzamanlı erişimden doğan tutarsızlık
record	Immutable, değer eşitlikli C# tipi
Reflection	Programın çalışırken kendi yapısını incelemesi
Repository	Veri erişimini koleksiyon gibi soyutlayan desen
Result Pattern	Hatayı exception yerine dönüş değeriyle taşıma
Rich Model	Veri ve davranışı birlikte tutan entity
sealed	Kalıtım yapılamayan sınıf
Serialization	Nesneyi JSON metnine çevirme
Short-circuit	Pipeline'da <code>next()</code> çağırarak zinciri kesme
Soft Delete	Kaydı silmeyip "silindi" olarak işaretleme
Specification	Sorgu kriterini nesneleştiren desen

Terim	Kısa açıklama
Unit of Work	Çok sayıda değişikliği tek atomik işlemde kaydetme
Use Case	Sistemin sunduğu tek bir anlamlı iş
Validator	Verinin şeklen doğruluğunu kontrol eden sınıf
Value Object	Kimliği olmayan, değeriyle tanımlanan nesne

Ek B — Sık Yapılan Hatalar

1. **Handler'a iş kuralı yazmak.** Kural entity'de olmalı ki nereden çağrılırsa çağrılсын çalışsın.
2. **Query'de `ToListAsync()` sonra `Select`.** Önce her şeyi çekip sonra ayıklamak, yavaş uygulamaların bir numaralı sebebidir. `Select` 'i sorgunun içine koy.
3. **Domain'e EF Core attribute'u koymak.** Fluent API kullan; domain temiz kalsın.
4. **Generic repository yazmak.** `DbContext` zaten repository. Spesifik, aggregate bazlı repository'ler yaz.
5. **`AsNoTracking()` unutmak.** Okuma sorgularında takip israftır.
6. **Üretimde `Database.Migrate()` çağırmak.** Çok kopyalı ortamda çakışır. CI/CD'de idempotent script çalıştır.
7. **Singleton içine Scoped enjekte etmek.** Captive dependency; `DbContext` ölür.
8. **Command'i mutable yapmak.** Pipeline'da değiştirilirse hata ayıklaması imkânsız hâle gelir.
9. **Her şeyi exception ile yönetmek.** Beklenen hatalar için `Result` kullan.
10. **Erken mikroservis.** Önce modüler monolit; sınırlar netleştğinde böl.
11. **`Publish` ile kritik iş yapmak.** Bir handler patlarsa diğerleri çalışmaz. Outbox kullan.
12. **Mimariyi dokümanda bırakmak.** Mimari testlerle CI'da zorla; yoksa altı ay içinde erir.